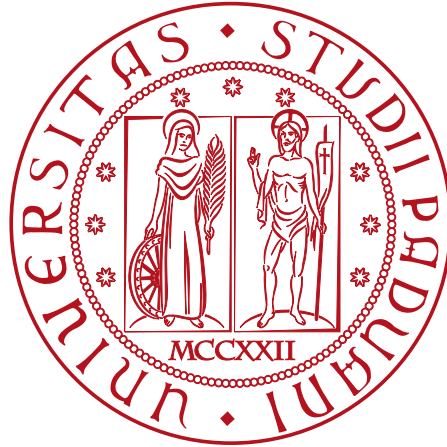


Università degli Studi di Padova

DIPARTIMENTO DI MATEMATICA "TULLIO LEVI-CIVITA"

CORSO DI LAUREA IN INFORMATICA



**Sviluppo di una applicazione in realtà virtuale
per la simulazione di procedure su macchinari**

Tesi di Laurea Triennale

Relatrice

Prof.ssa Ombretta Gaggi

Laureando

Andrea Precoma

Matricola

2068227

Sommario

Il presente documento descrive il lavoro svolto durante il periodo di stage, della durata di trecentoventi ore, dal laureando Andrea Precoma presso l'azienda Airlapp S.R.L.S.. L'attività ha avuto come scopo primario quello di ricerca e sviluppo nell'ambito delle interazioni uomo-ambiente virtuale. È stato richiesto di analizzare e approfondire i livelli di precisione e la versatilità del maggior numero di interazioni possibile. Con uno sguardo innovativo è stato richiesto inoltre di poter eseguire ogni tipo di operazioni anche con l'utilizzo delle mani, senza l'ausilio dei controller. Il tutto viene calato in uno scenario di formazione di personale addetto alla manutenzione e utilizzo di macchinari complessi. L'utente deve essere in grado di selezionare una procedura da applicare sul macchinario ed essere guidato nel suo completamento. La messa in pratica delle nozioni teoriche in un ambiente virtuale molto simile alla realtà dovrebbe migliorare l'acquisizione delle competenze da parte del nuovo personale.

Indice

1. Introduzione	1
1.1. Azienda	1
1.2. Progetto	1
1.2.1. Introduzione alla realtà virtuale	1
1.2.2. Stage	2
1.2.3. Obiettivi	2
1.2.4. Benefici attesi	3
1.3. Organizzazione del testo	3
2. Strumenti e tecnologie	5
2.1. Metodologia di lavoro	5
2.1.1. Kanban	5
2.2. Tecnologie utilizzate	6
2.2.1. Ambienti di sviluppo	6
2.2.1.1. Unity	6
2.2.1.2. Microsoft Visual Studio	6
2.2.2. Linguaggi e piattaforme di sviluppo	6
2.2.2.1. C#	6
2.2.2.2. .NET	7
2.2.3. Meta XR SDK	7
2.2.3.1. Interactor e interactable	8
2.2.3.2. One Grab e Two Grab Transformer	9
3. Ambiente di sviluppo Unity	10
3.1. Caratteristiche	10
3.1.1. Versione	10
3.2. Elementi base	10
3.3. Editor	11
3.4. Unity scripting	12
3.5. Configurazione di base	12
3.5.1. Project e build settings	12
3.5.2. Scena	12
4. Analisi dei requisiti	14
4.1. Diagrammi di sequenza	14
4.1.1. Passaggio automatico allo step successivo	14
4.2. Casi d'uso	14
4.2.1. UC1 - Movimento nello spazio utilizzando i controller	15
4.2.2. UC1.1 - Teletrasporto utilizzando i controller	15
4.2.3. UC1.2 - Traslazione utilizzando i controller	15
4.2.4. UC1.3 - Accovacciamento utilizzando i controller	16
4.2.5. UC2 - Traslazione di un oggetto lungo un piano avendolo afferrato	16
4.2.6. UC2.1 - Traslazione di un oggetto lungo un piano avendolo afferrato con la mano virtuale	16

4.2.7.	UC2.2 - Traslazione di un oggetto lungo un piano avendolo afferrato con il controller	17
4.2.8.	UC3 - Rotazione di un oggetto lungo un asse avendolo afferrato	17
4.2.9.	UC3.1 - Rotazione di un oggetto lungo un asse avendolo afferrato con la mano virtuale	17
4.2.10.	UC3.2 - Rotazione di un oggetto lungo un asse avendolo afferrato con il controller	18
4.2.11.	UC4 - Ancoraggio automatico di un oggetto in un preciso alloggio	18
4.2.12.	UC4.1 - Ancoraggio automatico di un oggetto in un preciso alloggio afferrato con la mano virtuale	19
4.2.13.	UC4.2 - Ancoraggio automatico di un oggetto in un preciso alloggio afferrato con il controller	19
4.2.14.	UC5 - Interazione con una interfaccia utente	19
4.2.15.	UC5.1 - Interazione con una interfaccia utente utilizzando la mano virtuale	20
4.2.16.	UC5.2 - Interazione con una interfaccia utente utilizzando il controller	20
4.2.17.	UC6 - Visualizzazione dati aziendali su interfaccia utente	20
4.2.18.	UC6.1 - Visualizzazione nome dell'azienda su interfaccia utente	21
4.2.19.	UC6.2 - Visualizzazione logo dell'azienda su interfaccia utente	21
4.2.20.	UC7 - Visualizzazione menu principale	21
4.2.21.	UC7.1 - Visualizzazione lista delle procedure nel menu principale	22
4.2.22.	UC7.1.1 - Visualizzazione singola procedura nella lista del menu principale	22
4.2.23.	UC7.1.1.1 - Visualizzazione nome della procedura nell'elemento della lista .	22
4.2.24.	UC8 - Visualizzazione dettagli procedura selezionata	23
4.2.25.	UC8.1 - Visualizzazione nome della procedura nei dettagli	23
4.2.26.	UC8.2 - Visualizzazione descrizione della procedura nei dettagli	24
4.2.27.	UC8.3 - Visualizzazione lista degli step nei dettagli della procedura	24
4.2.28.	UC8.3.1 - Visualizzazione singolo step nella lista dei dettagli della procedura	24
4.2.29.	UC8.3.1.1 - Visualizzazione nome dello step nell'elemento della lista	24
4.2.30.	UC9 - Avvio della procedura selezionata	25
4.2.31.	UC10 - Visualizzazione stato di avanzamento della procedura	25
4.2.32.	UC11 - Riavvio della procedura in esecuzione	26
4.2.33.	UC12 - Interruzione della procedura in esecuzione	26
4.2.34.	UC13 - Avvio dello step selezionato	27
4.2.35.	UC13.1 - Utilizzo esclusivo degli elementi dello step in esecuzione	27
4.2.36.	UC13.2 - Evidenziazione degli elementi dello step in esecuzione	27
4.2.37.	UC14 - Visualizzazione dettagli step in esecuzione	28
4.2.38.	UC14.1 - Visualizzazione nome dello step nei dettagli	28
4.2.39.	UC14.2 - Visualizzazione descrizione dello step nei dettagli	28
4.2.40.	UC15 - Riavvio dello step in esecuzione	29
4.2.41.	UC16 - Interruzione dello step in esecuzione	29
4.3.	Tracciamento dei requisiti	29
4.3.1.	Requisiti funzionali	30
4.3.1.1.	Requisiti obbligatori	30
4.3.1.2.	Requisiti desiderabili	31
4.3.1.3.	Requisiti facoltativi	32
4.3.2.	Requisiti di qualità	32
4.3.2.1.	Requisiti obbligatori	32
4.3.3.	Requisiti di vincolo	32
4.3.3.1.	Requisiti obbligatori	32
4.3.3.2.	Requisiti desiderabili	32

5. Progettazione	33
5.1. Architettura della scena	33
5.1.1. Ordine di esecuzione dei manager	33
5.1.2. Struttura del Camera Rig	33
5.2. Architettura delle procedure	34
5.2.1. Definizione con un automa a stati finiti non deterministici	34
5.2.2. Struttura delle procedure	35
5.3. Architettura del backend	37
5.4. Architettura della snap interaction	38
5.4.1. Progettazione e implementazione della snap interaction	38
5.5. Design pattern	40
5.5.1. Dependency injection pattern	40
5.5.1.1. Implementazione della dependency injection pattern	40
5.5.2. Singleton pattern	41
5.5.2.1. Implementazione del singleton pattern	41
5.5.3. Observer pattern	42
5.5.3.1. Implementazione dell'observer pattern	43
5.5.4. Template method pattern	43
5.5.4.1. Implementazione del template method pattern	43
5.5.5. Repository pattern	44
5.5.5.1. Implementazione del repository pattern	45
5.5.6. Command pattern	47
5.5.6.1. Implementazione del command pattern	47
6. Verifica e validazione	49
7. Conclusioni	50
7.1. Raggiungimento degli obiettivi	53
7.2. Soddisfacimento dei requisiti	54
7.3. Considerazioni personali	54
7.3.1. Aspetti implementati e possibili integrazioni	55
7.3.2. Conoscenze e competenze acquisite	55
8. Glossario	57
9. Acronimi e abbreviazioni	58
10. Sitografia	59

Elenco delle figure

Figura 1	Logo Airlapp S.R.L.S.	1
Figura 2	Visuale di un visore in realtà virtuale	2
Figura 3	Kanban board	5
Figura 4	Schema dell'interfaccia OpenXR	7
Figura 5	Confronto interazione Touch Hand Grab, Hand Grab e visuale reale	8
Figura 6	Unity Editor	11
Figura 7	Diagramma di sequenza del passaggio automatico allo step successivo	14
Figura 8	Diagramma dei casi d'uso UC1, UC1.1, UC1.2 e UC1.3	16
Figura 9	Diagramma dei casi d'uso UC2, UC2.1 e UC2.2	17
Figura 10	Diagramma dei casi d'uso UC3, UC3.1 e UC3.2	18
Figura 11	Diagramma dei casi d'uso UC4, UC4.1 e UC4.2	19
Figura 12	Diagramma dei casi d'uso UC5, UC5.1 e UC5.2	20
Figura 13	Diagramma dei casi d'uso UC6, UC6.1 e UC6.2	21
Figura 14	Diagramma dei casi d'uso UC7, UC7.1, UC7.1.1 e UC7.1.1.1	23
Figura 15	Diagramma dei casi d'uso UC8, UC8.1, UC8.2 e UC8.3	25
Figura 16	Diagramma dei casi d'uso UC8.3, UC8.3.1 e UC8.3.1.1	25
Figura 17	Diagramma dei casi d'uso UC9, UC10, UC11 e UC12	26
Figura 18	Diagramma dei casi d'uso UC13, UC13.1 e UC13.2	27
Figura 19	Diagramma dei casi d'uso UC14, UC14.1 e UC14.2	28
Figura 20	Diagramma dei casi d'uso UC15 e UC16	29
Figura 21	Automa che accetta tutte le procedure componibili con tre step	35
Figura 22	Esempio di un iniziatore di un elemento	36
Figura 23	Configurazione di un elemento di uno step dall'Editor	37
Figura 24	Confronto snap interaction con e senza override della componente orizzontale ...	39
Figura 25	Override della funzione Snap della classe Socket	39
Figura 26	Rimozione dei vincoli dell'oggetto riguardo la snap interaction	39
Figura 27	Configurazione degli script di Zenject dall'Editor	41
Figura 28	Singleton implementato in un sottotipo di MonoBehaviour	42
Figura 29	Binding dell'IoC di Zenject per singleton pattern	42
Figura 30	Preserve del costruttore per singleton pattern con Zenject	42
Figura 31	Dichiarazione dell'evento step completato nella classe Step	43
Figura 32	Iscrizione all'evento step completato nella classe Procedure	43
Figura 33	Diagramma della classe Socket	44
Figura 34	Diagramma della classe Step	44
Figura 35	Schema logico del repository pattern	45
Figura 36	Diagramma delle classi relative al repository pattern	46
Figura 37	Dati del file .json	46
Figura 38	Creazione del model	46
Figura 39	Creazione del view model	47
Figura 40	Diagramma di alcune classi relative al command pattern	48
Figura 41	Macchinario utilizzato inserito nella scena virtuale	50
Figura 42	Visuale del menu nella scena virtuale	51
Figura 43	Visuale del pannello informativo nella scena virtuale	51
Figura 44	Traslazione della console e della punta da intaglio	52
Figura 45	Rotazione della chiave a brugola e della manopola di precisione	52
Figura 46	Pressione del pulsante di emergenza e di una freccetta	53

Figura 47	Visuale completa della scena virtuale	55
-----------	---	----

Elenco delle tabelle

Tabella 1	Tracciamento dei requisiti funzionali obbligatori	30
Tabella 2	Tracciamento dei requisiti funzionali desiderabili	31
Tabella 3	Tracciamento dei requisiti funzionali facoltativi	32
Tabella 4	Tracciamento dei requisiti di qualità obbligatori	32
Tabella 5	Tracciamento dei requisiti di vincolo obbligatori	32
Tabella 6	Tracciamento dei requisiti di vincolo desiderabili	32
Tabella 7	Test di accettazione	49
Tabella 8	Tracciamento degli obiettivi raggiunti	53

Capitolo 1

Introduzione

1.1. Azienda



Figura 1: Logo Airlapp S.R.L.S.

Airlapp (*figura 1*) è un'azienda innovativa specializzata nello sviluppo di applicazioni 3D, realtà aumentata e realtà virtuale. Opera principalmente su progetti con una forte percentuale di ricerca e sviluppo dedicati alle grandi realtà industriali.

Tra i progetti più significativi realizzati dall'azienda vi è il configuratore web 3D sviluppato per Parlanti Roma, che consente ai clienti di personalizzare stivali da equitazione scegliendo modello, materiale, colore e accessori. Per De Angeli Prodotti, leader nella produzione di conduttori per il trasporto dell'energia, è stato creato un configuratore web per facilitare la scelta del prodotto più adeguato alle esigenze dell'utente.

Airlapp ha ricevuto riconoscimenti per la sua innovazione, tra cui il Premio Innovazione SMAU nel 2018 e nel 2019, rispettivamente per una piattaforma VR per il gruppo Bedeschi e per un configuratore 3D di elmi da lavoro sviluppato in collaborazione con Sicor. L'età media è di 30 anni, l'ambiente è stimolante, amichevole e sempre aperto alle nuove tecnologie.

1.2. Progetto

1.2.1. Introduzione alla realtà virtuale

La realtà virtuale è una tecnologia che permette di immergersi in ambienti digitali tridimensionali e interattivi, simulando situazioni reali o immaginarie attraverso l'uso di computer e dispositivi specifici come visori, controller, telecamere e sensori di movimento. Questa immersione è resa possibile dal coinvolgimento simultaneo di più sensi, principalmente vista e udito, e in alcuni casi anche tatto. Questa combinazione crea una sensazione di presenza che inganna il cervello facendogli percepire l'ambiente virtuale come reale. Un esempio della visuale da un visore in realtà virtuale è mostrato nella *figura 2*.

La realtà virtuale nasce da esperimenti già a partire dagli anni '60 con dispositivi come il Sensorama di Morton Heilig e il visore "La Spada di Damocle" di Ivan Sutherland; si è evoluta poi fino a diventare una tecnologia piuttosto matura e diffusa utilizzata in ambiti che vanno dal videoludico all'educazione e alle simulazioni professionali.

La realtà virtuale differisce dalla realtà aumentata in quanto la seconda non sostituisce il mondo reale, bensì lo arricchisce sovrapponendo elementi digitali (testi, immagini, animazioni)

all'ambiente fisico circostante in tempo reale. In questo modo l'utente mantiene il contatto con la realtà che viene "aumentata" da informazioni o oggetti virtuali.

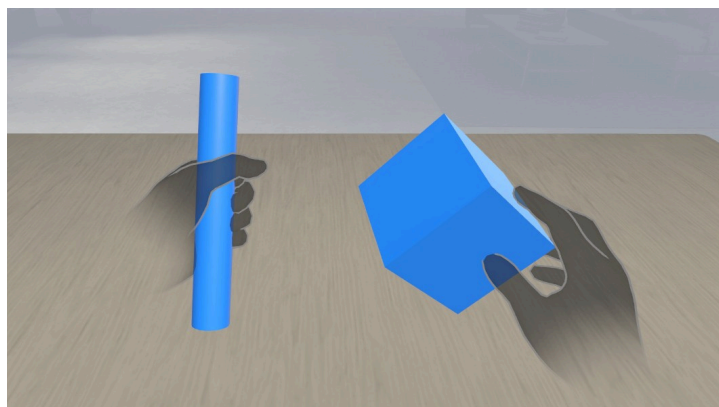


Figura 2: Visuale di un visore in realtà virtuale

1.2.2. Stage

L'azienda possiede strumenti per sviluppare applicazioni in realtà virtuale e aumentata, in particolare un visore Meta Quest 3. Lo stage si propone di utilizzare tali strumenti per realizzare un simulatore in realtà virtuale per lo svolgimento di procedure su macchinari, ad esempio attrezzaggio e manutenzione.

Questa proposta nasce dall'esigenza di efficientare la formazione all'utilizzo di macchinari molto complessi, in particolare in industrie dove il personale qualificato è ridotto e risulta quindi difficile occupare diverse risorse per l'istruzione dei nuovi addetti. L'applicazione non andrebbe a sostituire la formazione del nuovo personale bensì a precederla in modo tale da preparare i nuovi operatori all'esperienza pratica.

Il simulatore deve essere composto da un ambiente 3D verosimile a quello dove gli operatori andranno ad agire. In questo ambiente devono essere messi a disposizione degli *scenari operativi* quali spostarsi nello spazio, afferrare oggetti, interagire con le interfacce utente. Avendo consapevolezza dell'utenza target tutte le funzionalità devono essere completabili con l'utilizzo dei joypad. Da esperienze passate l'azienda ha infatti notato che un'utenza non propriamente giovane riscontra maggiori difficoltà a interagire con un ambiente virtuale utilizzando unicamente le proprie mani. La totalità delle interazioni tuttavia devono poter essere svolte anche sfruttando il tracciamento delle mani come alternativa ai joypad.

1.2.3. Obiettivi

Lo stage pone come obiettivo quello di approfondire le tipologie e i meccanismi di interazione tra utente e realtà virtuale. Si privilegia quindi la precisione delle stesse a dispetto del realismo con il quale si svolgono tali azioni. Una modalità di interazione indagata è stata ad esempio l'utilizzo di una chiave a brugola: questa deve essere afferrata, inserita nell'apposito alloggiamento, ruotata lungo uno specifico asse mantenendola bloccata nell'alloggio e successivamente sfilata.

Il principale aspetto richiesto dall'azienda è stato quindi quello di costruire una procedura composta da diverse tipologie di interazione, ad esempio traslazione e rotazione di oggetti, pressione di pulsanti, impugnatura di attrezzatura specifica. È stata concessa molta libertà nella scelta degli scenari operativi proprio per lasciare spazio allo studio e alla ricerca.

In supporto all'esperienza pratica è stato richiesto di affiancare delle interfacce utente. Queste devono svolgere funzioni informative ma devono anche essere oggetto di interazione. Avendo

sempre a che fare con schermi piatti la proiezioni di interfacce utente in un ambiente virtuale rende l'esperienza più immersiva e interessante.

1.2.4. Benefici attesi

La formazione professionale in alcuni ambiti risulta più efficace se effettuata in prima persona. Non sempre tuttavia è agibile questa possibilità limitando quindi l'insegnamento al personale. Approcciando un ambiente in realtà aumentata è possibile simulare con adeguata precisione delle circostanze reali nelle quali l'utente viene immerso.

Per un insegnamento coinvolgente è fondamentale sfruttare molteplici scenari operativi. Il progetto si concentra sulle principali interazioni tra utente e ambiente virtuale:

- * interazione diretta con i modelli 3D e l'ambiente, ad esempio afferrare oggetti e spostarli;
- * interazione diretta con interfacce di macchina simulate, ad esempio pulsanti;
- * interazione con quiz a risposta multipla.

Per migliorare le sensazioni restituite dall'ambiente virtuale in modo che sembri il più verosimile possibile vengono usate delle *Application Programming Interface (API)_G* per sostituire i joystick con il tracciamento delle mani. Con l'ausilio delle telecamere installate nel visore si riesce a distinguere i movimenti delle mani e delle dita. In questo modo, con una buona progettazione delle azioni e dell'ambiente, è possibile interagire con gli elementi aumentando il realismo degli scenari.

1.3. Organizzazione del testo

Il presente documento è organizzato nella seguente struttura:

- * **capitolo 1** (presente) - introduzione; sono esposte le principali informazioni riguardanti l'azienda e il progetto proposto;
- * **capitolo 2** - strumenti e tecnologie; sono analizzate in dettaglio le tecnologie utilizzate nel progetto;
- * **capitolo 3** - ambiente di sviluppo Unity; sezione dedicata all'ambiente di sviluppo utilizzato per la modellazione dell'ambiente virtuale perché piuttosto complesso;
- * **capitolo 4** - analisi dei requisiti; vengono analizzati gli aspetti richiesti dal progetto con un diagramma delle sequenze, casi d'uso e i relativi requisiti;
- * **capitolo 5** - progettazione; vengono descritte in dettaglio le scelte architetturelle dell'applicazione seguite dai relativi metodi implementativi;
- * **capitolo 6** - verifica e validazione; sono indicati i metodi di test e tracciati gli esiti;
- * **capitolo 7** - conclusioni; vengono analizzati gli aspetti implementati in relazione agli obiettivi iniziali, tracciati i requisiti rispettati ed esposte delle considerazioni personali relative al progetto e all'intera esperienza.

Riguardo la stesura del testo, relativamente al documento sono state adottate le seguenti convenzioni tipografiche:

- * nel formato .pdf i termini sottolineati sono interattivi e conducono a una parte del documento specifica;
- * i termini ambigui o di uso non comune menzionati vengono definiti nel *glossario*, situato alla fine del presente documento;

- * per la prima occorrenza dei termini riportati nel glossario viene utilizzata la seguente nomenclatura: *esempio*_G;
- * gli acronimi e le abbreviazioni vengono definite nella *sezione omonima*, situata in seguito al glossario;
- * per la prima occorrenza degli acronimi o abbreviazioni riportate nella sezione omonima viene utilizzata la seguente nomenclatura: *Esempio di Acronimo (EdA)*_G;
- * i nomi di file, classi o estensioni vengono scritte in carattere monospace, quindi con il seguente stile: NomeClasse.

Le fonti ritenute importanti ai fini di una corretta comprensione del documento sono riportate nella *sitografia*, situata in seguito alla sezione degli acronimi e abbreviazioni.

Capitolo 2

Strumenti e tecnologie

2.1. Metodologia di lavoro

Per un lavoro ben organizzato è essenziale scegliere una corretta metodologia che aderisca il più possibile ai componenti del team. Questi infatti cooperano nella maniera più efficiente possibile quando le attività sono distinte, assegnate e collocate in uno spazio temporale, fissando quindi delle scadenze. Si deve privilegiare il lavoro asincrono perché più produttivo, intervallato tuttavia da brevi riunioni sincrone con l'obiettivo di riallineare il lavoro svolto e da svolgere.

2.1.1. Kanban

La metodologia di gestione del flusso di lavoro adottata in questo progetto è la Kanban. Aiuta a visualizzare il lavoro dividendo le attività tipicamente in tre sezioni, esposte graficamente dalla *figura 3*:

- * **TO DO** - attività da svolgere, sono assegnate ai membri del gruppo che dovranno portarle a termine entro una certa data. Possono avere delle priorità diverse in modo da avere un quadro più dettagliato dell'importanza delle attività;
- * **DOING** - attività in corso, sono state prese in carico dai relativi componenti del gruppo;
- * **DONE** - attività svolte, sono state concluse. Il concetto di "DONE" deve essere ben definito così da chiarire quando una attività può essere considerata conclusa con successo. Tipicamente un task viene definito chiuso quando ha soddisfatto i requisiti richiesti superando i relativi test.

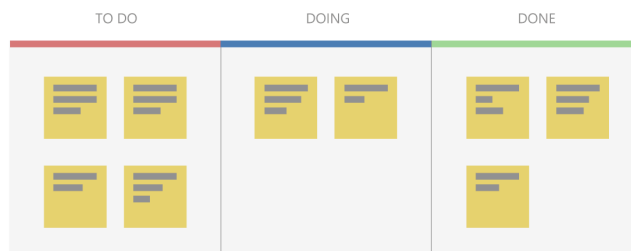


Figura 3: Kanban board

L'azienda adotta la metodologia Kanban perché agile e visivamente efficace. In ufficio è infatti affissa una lavagna divisa nelle tre colonne precedentemente descritte. Questa viene usata principalmente per le attività più generiche, ad esempio "concludere la funzionalità X" e per riallineare il team sull'avanzamento del lavoro. Il gruppo poi scompone queste attività in task minori e le organizza con il sistema di ticketing Jira.

2.2. Tecnologie utilizzate

Nella seguente sezione vengono elencate e descritte le tecnologie adottate in riflesso di quanto studiato durante il periodo di formazione e precedenti esperienze.

2.2.1. Ambienti di sviluppo

Per lo svolgimento del progetto di stage è stato scelto Unity per la creazione dell'ambiente virtuale, Microsoft Visual Studio per lo sviluppo del codice.

2.2.1.1. Unity

Originariamente nato come game engine, Unity ha esteso i suoi campi integrando tool per sviluppare scenari interattivi in realtà virtuale e aumentata. Offre una elevata compatibilità per numerose piattaforme di rilascio per gli applicativi. Nel progetto viene sfruttato come piattaforma di modellazione per l'ambiente 3D, sviluppo dell'interfaccia utente e interazione con gli elementi.

Data l'importanza ricoperta all'interno del progetto, per una descrizione dettagliata dell'ambiente e delle integrazioni con la realtà virtuale si rimanda al *capitolo 3*.

2.2.1.2. Microsoft Visual Studio

Più comunemente chiamato Visual Studio è un *Integrated Development Environment (IDE)*_G sviluppato da Microsoft che supporta diversi linguaggi di programmazione tra cui C#, C++ e Visual Basic. Fornisce un approccio *Rapid Application Development (RAD)*_G volto ad aumentare la produttività mettendo a disposizione alcuni strumenti quali *IntelliSense*_G e un designer visuale di finestre.

2.2.2. Linguaggi e piattaforme di sviluppo

Per lo svolgimento del progetto di stage è stato scelto il linguaggio C# in combinazione con la piattaforma .NET in quanto supportata nativamente da Unity e Visual Studio.

2.2.2.1. C#

Sviluppato da Microsoft, C# è il linguaggio orientato agli oggetti che meglio si adatta a un progetto di realtà virtuale costruito su Unity dato il suo supporto nativo con la piattaforma .NET. Questa infatti facilita le operazioni di compilazione, debug e test, monitoraggio codice e distribuzione su molteplici piattaforme.

La sintassi di questo linguaggio è ispirata a C++ e Java. Di seguito vengono elencate le principali funzionalità sfruttate nel progetto:

- * **proprietà** - campo di una classe che fornisce personalizzazioni riguardo i metodi di accesso in lettura e scrittura e la visibilità della stessa. Si possono specificare operazioni da eseguire all'utilizzo della proprietà quali controlli o assegnazioni di altre variabili;
- * **generics** - offrono flessibilità nella progettazione di classi e metodi rinviando la specifica dei parametri di tipo fino al momento in cui si usa effettivamente la classe o il metodo;
- * **nullable type** - permette di attribuire il valore "null" alle variabili che non fanno riferimento ad alcun oggetto allocato in memoria;
- * **garbage collection** - permette il recupero automatico della memoria occupato da oggetti inutilizzati o irraggiungibili;

- * **lambda expression** - funzioni anonime che forniscono supporto a tecniche di programmazione funzionale;
- * **operazioni asincrone** - consentono operazioni utili per sistemi distribuiti o per funzioni eseguibili in più *thread* di esecuzione.

2.2.2.2. .NET

I programmi scritti in C# vengono eseguiti sulla piattaforma di sviluppo .NET la quale coniuga un sistema di esecuzione virtuale chiamato *Common Language Runtime (CLR)*_G e un insieme di librerie di classi. Il CLR è l'implementazione di Microsoft della *Common Language Infrastructure (CLI)*_G, uno standard internazionale che costituisce la base di creazione di ambienti di esecuzione e sviluppo nei quali linguaggi e librerie operano insieme. Il codice scritto in C# viene compilato in un linguaggio intermedio (*Intermediated Language (IL)*_G) conforme alle specifiche della CLI il quale, insieme alle varie risorse, viene archiviato in un *assembly*_G. Quando il programma scritto in C# viene eseguito, l'assembly viene caricato nel CLR che effettua una compilazione *Just In Time (JIT)*_G per convertire il codice IL in linguaggio macchina.

Il codice IL prodotto dal compilatore è conforme alla *Common Type Specification (CTS)*_G e può interagire con codice prodotto dalle versioni .NET di F#, Visual Basic e C++. .NET include inoltre librerie estese organizzate in namespaces che forniscono svariate funzionalità tra cui la gestione dei file in input e output, la manipolazione di stringhe, framework per applicazioni web.

2.2.3. Meta XR SDK

Il *Software Development Kit (SDK)*_G distribuito da Meta per Unity mette a disposizione moduli per facilitare le interazioni con l'ambiente virtuale. In particolare vengono sfruttati i "building blocks", *GameObject*_G già strutturati per svolgere alcune funzionalità. I principali utilizzati nel progetto vengono elencati nella *sezione 3.5.2* apposita.

Per interfacciarsi con il visore è stato scelto lo standard OpenXR. Questo fornisce un set di API che permette di eseguire l'applicazione su molteplici dispositivi adibiti alla realtà virtuale e aumentata, come mostrato in *figura 4*. Meta aggiorna la SDK piuttosto rapidamente per mettere a disposizione nuove funzionalità o efficientare le precedenti, quindi sebbene siano uscite nuove versioni durante il progetto è stata mantenuta la 76.0.0 per tutta la durata dello stage.

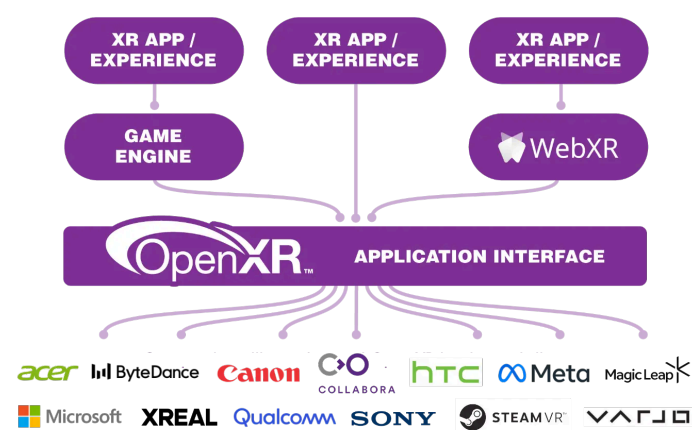


Figura 4: Schema dell'interfaccia OpenXR

2.2.3.1. Interactor e interactable

Il progetto verte sull'analisi della loro varietà e precisione delle interazioni. È stato quindi fondamentale comprendere a fondo le relazioni tra i due componenti principali: gli interactor e gli interactable. I primi sono gli elementi che agiscono attivamente con gli oggetti, i secondi vengono attivati dai primi in seguito ad una selezione. Ogni tipo di interactor può interagire solamente con il corrispettivo tipo di interactable. Di seguito vengono presentati quelli utilizzati nel progetto:

- * **Hand Grab** - interazioni riguardanti afferrare oggetti con le mani. Non sfrutta la *mesh* dell'oggetto per permettere una presa maggiormente verosimile (sarebbe stato necessario utilizzare il componente Touch Hand Grab), bensì le mani virtuali possono penetrare l'oggetto. Per gli scopi del progetto è stata preferita questa scelta perché ritorna un riscontro migliore all'utente. Nel caso del Touch Hand Grab infatti per afferrare un oggetto bisogna comunque unire pollice e indice, ma le dita della mano virtuale assumono una posa diversa a seconda della forma dell'oggetto. La *figura 5* dimostra un confronto tra le due interazioni e la realtà;

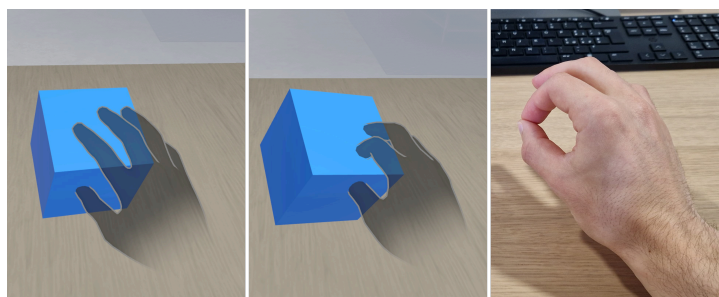


Figura 5: Confronto interazione Touch Hand Grab, Hand Grab e visuale reale

- * **Grab** - interazioni riguardanti afferrare oggetti con i controller. Sulla parte frontale del joystick viene disposto virtualmente un rilevatore che se attivato interagisce con l'oggetto;
- * **Ray** - interazioni riguardanti le interfacce grafiche. Viene proiettato un raggio dalla mano o dal controller che collide con il primo elemento grafico che intercetta;
- * **Poke** - interazioni riguardanti la pressione di oggetti con rilascio. Il principale utilizzo è quello dei pulsanti che rilevano la pressione della mano o del controller ma che si sollevano al loro rilascio;
- * **Snap** - interazioni riguardanti il posizionamento automatico dell'oggetto in una locazione specifica. Nel progetto è stato implementato personalmente perché quello nativo permette il posizionamento solo al rilascio dell'oggetto sebbene questo sia vicino all'alloggio di destinazione. Dovendo progettare interazioni più complesse è stato necessario ridefinirla e arricchirla. Il lato progettuale e implementativo è descritto nella *sezione 5.4*.

Ciascun interactable deve possedere inoltre due componenti fondamentali: un Rigidbody che attribuisce le logiche della fisica e un Collider che definisce una superficie con la quale collidere. È importante impostare il Collider come trigger cosicché non si scontri con altri oggetti ma rilevi le penetrazioni con altri Collider definiti trigger, in particolare quelli degli interactor.

Per permettere ad un oggetto di essere afferrato è necessario aggiungere un ulteriore componente chiamato Grabbable. Questo attribuisce le caratteristiche principali per consentire l'interazione. Due tra le impostazioni più importanti sono le seguenti: “transfer on second selection” per afferrare l'oggetto con una mano se questo è già tenuto dall'altra, “kinematic while selected” per rendere l'oggetto non affetto dalla fisica e potersi quindi muovere insieme all'interactor.

2.2.3.2. One Grab e Two Grab Transformer

La maggior parte delle interazioni coinvolgono dei limiti nei movimenti dell'oggetto. Si prenda come esempio una porta scorrevole che deve essere afferrata ma può essere mossa solamente lungo un asse specifico e per una certa distanza. A questo proposito è stato rilevante comprendere il funzionamento dei Transformer, componenti che limitano le operazioni di traslazione, rotazione e scalabilità degli oggetti. Si distinguono in One Grab e Two Grab, utilizzati se l'oggetto è afferrato rispettivamente da una o due mani. Di seguito vengono elencati i principali utilizzati nel progetto:

- * **One Grab Translate Transformer** - permette di limitare i movimenti lungo i tre assi dello spazio. Se vengono omessi l'oggetto è libero di muoversi ed è come non avesse il componente;
- * **One Grab Rotate Transformer** - consente la rotazione lungo un solo asse con eventuali restrizioni sull'ampiezza dell'angolo;
- * **Free Grab Transformer** - permette di limitare lo spostamento, la rotazione e la scalabilità. Viene utilizzato nella personale implementazione della snap interaction quando l'oggetto viene rimosso dal proprio alloggio e deve quindi poter essere manipolato a piacimento (non vengono infatti posti vincoli). Nella sezione 5.4 è spiegata nel dettaglio la scelta progettuale riguardante questo componente.

Capitolo 3

Ambiente di sviluppo Unity

La seguente sezione espone gli aspetti principali di Unity, riservando particolare attenzione a quelli caratterizzanti il progetto di stage. Avendo avuto una esperienza quadriennale con questo IDE, seppur in ambito non professionale, l'apprendimento di nuovi metodi di sviluppo è stato pressoché immediato. Viene quindi dato più spazio agli aspetti approfonditi nei primi giorni di formazione come la configurazione dei building blocks di Meta.

3.1. Caratteristiche

Unity è un motore grafico che consente lo sviluppo di numerosi contenuti interattivi come videogiochi, animazioni 3D, applicazioni in realtà virtuale e aumentata. Si basa sulle associazioni tra componenti e *script*: i secondi attribuiscono i comportamenti logici ai primi. Gli oggetti, definiti propriamente con il termine “GameObject”, vengono composti con diversi componenti, ad esempio quello per le interazioni con la fisica, per le animazioni, per i suoni; gli script possono interagire con i componenti attribuendo un comportamento all'oggetto. I GameObject sono interamente gestiti dall'IDE che include un editor, un rendering della scena in tempo reale, un simulatore per la fisica e un sistema di animazione.

3.1.1. Versione

Dal 2024 Unity ha pubblicato la nuova serie di versioni 6000 con l'intento di migliorare in modo significativo le caratteristiche sfruttate per videogiochi multi-piattaforma, simulazioni e applicazioni in realtà virtuale e aumentata. Lo stesso sito ufficiale di Meta per le SDK consiglia di utilizzare una di queste nuove versioni perché più performanti. Per questo motivo è stata adottata la versione 6000.0.47f1 *Long-Term Support (LTS)*.

3.2. Elementi base

Alcuni concetti fondamentali sui quali si basa Unity sono stati anticipati nella *sezione 3.1*. Di seguito vengono descritti tutti gli elementi coi quali si può completare la quasi totalità di una applicazione utilizzando questo engine:

- * **componente** - proprietà che si aggiungono ai GameObject in modo da attribuirgli dei comportamenti. L'approccio di sviluppo di Unity infatti è incentrato sulla composizione, si vanno cioè a comporre gli oggetti combinando componenti che assegnano diverse caratteristiche ciascuno;
 - **script** - un particolare tipo di componente costituito da codice, tipicamente C#. Necessario per personalizzare comportamenti logici;
- * **GameObject** - elemento nella scena, visibile o nascosto, costituito da almeno un componente che attribuisce le coordinate nello spazio, la rotazione e le dimensioni. Si può mantenere vuoto e utilizzare come contenitore per altri GameObject, oppure arricchire con componenti diversi per aumentarne le funzionalità;

- ***prefab*** - GameObject salvato nelle cartelle del progetto e quindi indipendente dalla scena. Usato come “template” può essere riutilizzato in più scene o generare delle varianti, cioè altri prefab che condividono dei GameObject o componenti. Le modifiche apportate ai prefabbricati vengono propagate a tutte le istanze nelle scene;
- * **asset** - risorse utilizzate per creare l'applicazione, ad esempio effetti audio e video, texture, modelli 2D e 3D;
- * **scene** - ambiente di lavoro che contiene i GameObject, solitamente una o più camere, delle luci e elementi di configurazione della scena. Con l'utilizzo di appositi *package* si possono combinare più scene in modo da alleggerire il caricamento delle risorse durante l'esecuzione dell'applicazione.

3.3. Editor

Il motore grafico mette a disposizione numerosi pannelli e tool per configurare l'ambiente 2D o 3D. Un esempio di disposizione dell'Editor con i pannelli principali è mostrato in *figura 6*. Di seguito vengono brevemente descritti le principali finestre utilizzate:

- * **scene** - espone l'ambiente di lavoro, consentendo di muoversi e interagire all'interno dello spazio con l'utilizzo di mouse e tastiera e visualizzarne gli elementi;
- * **game** - permette di visualizzare la schermata dell'applicazione una volta in esecuzione. Con un tasto si attiva la modalità “play” la quale simula l'avvio e l'esecuzione dell'applicazione pur rimanendo nell'Editor;
- * **hierarchy** - permette di visualizzare la gerarchia dei GameObject all'interno in forma testuale. Consente inoltre di modificare le relazioni padre-figlio degli elementi;
- * **inspector** - espone i componenti del GameObject selezionato, permettendo di aggiungere, togliere e modificare le proprietà;
- * **project** - espone la struttura delle cartelle e file del progetto in un formato simile a quella della gerarchia della scena. Permette quindi di navigare tra gli asset e fornisce la funzionalità “drag and drop” molto comoda per importare alcuni tipi di risorse;
- * **console** - fornisce i messaggi di informazione dell'Editor, debug, warning e errori sia nella fase di esecuzione dell'applicazione che in quella di sviluppo.

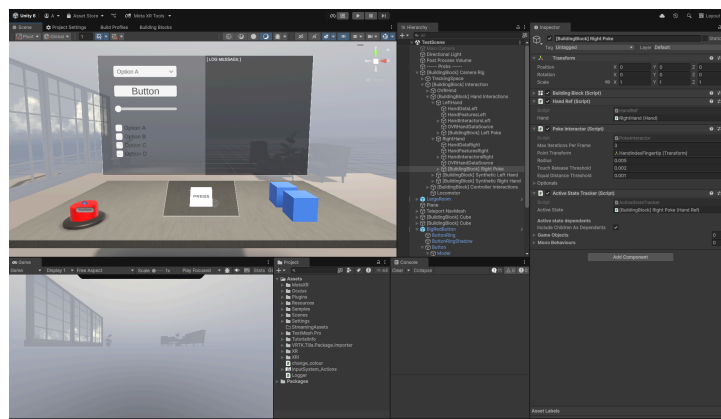


Figura 6: Unity Editor

Una funzione di rilevante importanza è stata ricoperta dal package Android Logcat. Questo fornisce un pannello simile alla console di Unity ma con funzionalità specifiche per analizzare i messaggi prodotti durante l'esecuzione delle applicazioni su piattaforma Android. Non avendo disposto di un computer con scheda grafica dedicata non è stato possibile usufruire del simulatore di Meta integrato con Unity. Questo avrebbe permesso di visualizzare i messaggi prodotti dall'applicazione durante l'esecuzione sul visore.

3.4. Unity scripting

Come anticipato gli script permettono di definire la logica dell'applicazione. Questi sono tipicamente legati ai GameObject e richiedono quindi di essere estesi alla classe `MonoBehaviour` integrata in Unity. Questa contiene inoltre dei metodi built-in chiamati automaticamente in certi stati dell'applicazione. Di seguito si elencano i principali metodi utilizzati:

- * **Awake()** - primo metodo chiamato all'avvio dell'applicazione;
- * **Start()** - quarto metodo invocato per l'inizializzazione dall'avvio;
- * **Update()** - funzione chiamata all'inizio di ogni frame;
- * **OnDestroy()** - ultima funzione invocata alla distruzione del GameObject.

Non è tuttavia obbligatorio associare uno script a un elemento della scena per poterlo eseguire. Si può sfruttare a pieno la programmazione ad oggetti creando classi con personali costruttori e istanziandoli dove necessario. Le classi infatti estese alla `MonoBehaviour` sono prive del costruttore e vengono istanziate unitamente al GameObject associato, condividendo quindi lo stesso ciclo di vita.

3.5. Configurazione di base

Per poter avviare l'applicazione su un visore è necessario configurare l'ambiente di Unity con alcuni package e la scena principale con dei GameObject specifici. La seguente sezione si occupa di descrivere in linea generale i requisiti per poter lavorare in un ambiente di realtà virtuale usando Unity e il visore Meta Quest 3.

3.5.1. Project e build settings

Innanzitutto bisogna assicurarsi di impostare la piattaforma di build in Android, sistema operativo del visore. Alla fine del processo infatti viene reso disponibile un `.apk` che può essere installato ed eseguito sul dispositivo.

In seguito è stato installato l'asset "Meta XR All-in-One SDK" di Meta direttamente dall'asset store di Unity. Questo contiene già tutte le dipendenze per sviluppare una applicazione in realtà virtuale fornendo anche dei prefab e i building blocks descritti alla [sezione 2.2.3](#). Infine è stato installato l'"XR Plug-in Management" e i relativi package di OpenXR, interfaccia base per tutti i dispositivi di realtà virtuale.

3.5.2. Scena

La scena deve contenere i seguenti elementi affinché si possa visualizzare l'ambiente sul visore, adoperare i controller e le mani per interagire. La quasi totalità di questi sono building blocks:

- * **Camera Rig** - elemento principale che comprende la camera per registrare la visuale del visore;

- * **Controller Tracking** - moduli che permettono il tracciamento dei joypad e la loro visualizzazione;
- * **Virtual Hands** - moduli che permettono il tracciamento delle mani e la loro visualizzazione;
- * **Teleport** - modulo che consente lo spostamento nello spazio attraverso il teletrasporto;
- * **Locomotor** - prefab che aggiunge funzionalità nel movimento usando i controller, quali lo spostamento sul piano trasversale, la corsa, l'accovacciamento e il salto;
- * **interactions** - moduli per l'interazione con gli elementi nella scena, in particolare:
 - **Hand Grab Interaction** - permette di afferrare gli oggetti con le mani;
 - **Grab Interaction** - permette di afferrare gli oggetti con i controller;
 - **Poke Interaction** - consente di premere degli oggetti sia con le mani che con i controller, ad esempio dei pulsanti;
 - **Ray Interaction** - consente di interagire a distanza con degli elementi grafici sia con le mani che con i controller utilizzando dei raggi, ad esempio per interagire con l'interfaccia utente.

Capitolo 4

Analisi dei requisiti

Nella seguente sezione sono analizzati i requisiti dell'applicazione. Questi vengono modellati in casi d'uso se comprendenti azioni dall'esterno verso il sistema, in diagrammi di sequenza se inerenti al comportamento del sistema stesso.

4.1. Diagrammi di sequenza

I diagrammi di sequenza costituiscono un metodo grafico che modella il flusso del sistema tramite attività. All'inizio della sequenza viene generato un gettone o token, questo passa attraverso le attività che possono moltiplicarlo, consumarlo o direzionarlo. Al termine di ogni percorso o sequenza bisogna che tutti i gettoni siano stati consumati.

4.1.1. Passaggio automatico allo step successivo

Tutte le procedure sono costituite da step. Al completamento di uno il sistema carica in automatico quello successivo procedendo così allo svolgimento della procedura. Se si è completato lo step finale la procedura è considerata conclusa. La *figura 7* illustra il diagramma di sequenza.

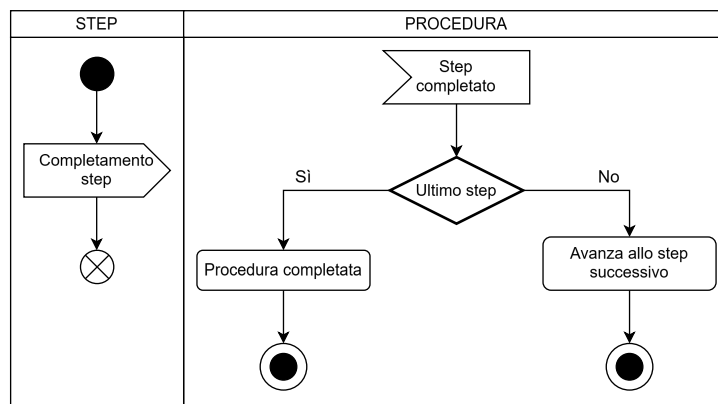


Figura 7: Diagramma di sequenza del passaggio automatico allo step successivo

4.2. Casi d'uso

L'analisi dei requisiti ha prodotto unicamente casi d'uso nei quali l'attore principale è l'utente in quanto è l'unico a interagire attivamente con il sistema. Ogni caso d'uso è accompagnato da un diagramma che espone graficamente le sue caratteristiche. Ciascuno di essi possiede la seguente struttura:

- * **diagramma di riferimento** - il diagramma che descrive graficamente il caso d'uso;
- * **attore principale** - l'attore principale del caso d'uso, unicamente l'utente;
- * **precondizioni** - lo stato del sistema prima che venga eseguito il caso d'uso;

- * **postcondizioni** - lo stato del sistema dopo che è stato eseguito il caso d'uso;
- * **scenario principale** - la sequenza delle azioni da compiere per verificare il caso d'uso;
- * **inclusioni** - eventuali casi d'uso che vengono inclusi, vengono sempre verificati al compimento del caso d'uso;
- * **generalizzazioni** - eventuali casi d'uso che vengono generalizzati, sono mutuamente esclusivi quindi uno solo viene eseguito.

4.2.1. UC1 - Movimento nello spazio utilizzando i controller

- * **diagramma di riferimento:** *figura 8*;
- * **attore principale:** utente;
- * **precondizioni:** l'utente adopera i controller per muoversi nello spazio virtuale;
- * **postcondizioni:** l'utente si sposta nello spazio virtuale;
- * **scenario principale:**
 1. l'utente utilizza i controller per muoversi;
 2. la rappresentazione virtuale dell'utente viene spostata nello spazio;
- * **generalizzazioni:**
 1. Teletrasporto utilizzando i controller (*UC1.1*);
 2. Traslazione utilizzando i controller (*UC1.2*);
 3. Accovacciamento utilizzando i controller (*UC1.3*).

4.2.2. UC1.1 - Teletrasporto utilizzando i controller

- * **diagramma di riferimento:** *figura 8*;
- * **attore principale:** utente;
- * **precondizioni:** l'utente utilizza lo stick del controller di destra;
- * **postcondizioni:** l'utente si sposta nello spazio virtuale;
- * **scenario principale:**
 1. l'utente sposta in avanti lo stick del controller di destra;
 2. viene visualizzata la proiezione del punto in cui l'utente verrà teletrasportato;
 3. l'utente rilascia lo stick del controller di destra e viene teletrasportato nel punto precedentemente visualizzato.

4.2.3. UC1.2 - Traslazione utilizzando i controller

- * **diagramma di riferimento:** *figura 8*;
- * **attore principale:** utente;
- * **precondizioni:** l'utente utilizza lo stick del controller di sinistra;
- * **postcondizioni:** l'utente si sposta nello spazio virtuale;
- * **scenario principale:**
 1. l'utente sposta in qualunque direzione lo stick del controller di sinistra;
 2. l'utente viene traslato sul piano orizzontale verso la direzione puntata dallo stick del controller.

4.2.4. UC1.3 - Accovacciamento utilizzando i controller

- * **diagramma di riferimento:** *figura 8*;
- * **attore principale:** utente;
- * **precondizioni:** l'utente utilizza lo stick del controller di destra;
- * **postcondizioni:** l'utente si accovaccia o si alza nello spazio virtuale;
- * **scenario principale:**
 1. l'utente sposta indietro lo stick del controller di destra;
 2. la visuale dell'utente viene immediatamente abbassata se questa era all'altezza originale;
 3. la visuale dell'utente viene immediatamente riportata all'altezza originale se questa era in posizione di accovacciamento.

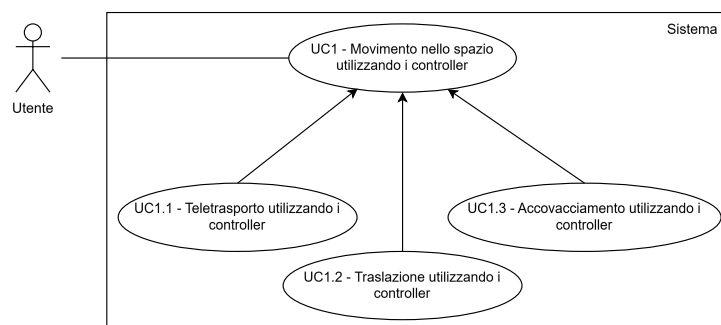


Figura 8: Diagramma dei casi d'uso UC1, UC1.1, UC1.2 e UC1.3

4.2.5. UC2 - Traslazione di un oggetto lungo un piano avendolo afferrato

- * **diagramma di riferimento:** *figura 9*;
- * **attore principale:** utente;
- * **precondizioni:** l'utente afferra l'oggetto;
- * **postcondizioni:** l'oggetto è traslato lungo un piano;
- * **scenario principale:**
 1. l'utente afferra l'oggetto;
 2. l'utente spostando la mano nello spazio fa muovere l'oggetto lungo un piano;
- * **generalizzazioni:**
 1. Traslazione di un oggetto lungo un piano avendolo afferrato con la mano virtuale (*UC2.1*);
 2. Traslazione di un oggetto lungo un piano avendolo afferrato con il controller (*UC2.2*).

4.2.6. UC2.1 - Traslazione di un oggetto lungo un piano avendolo afferrato con la mano virtuale

- * **diagramma di riferimento:** *figura 9*;
- * **attore principale:** utente;
- * **precondizioni:** l'utente afferra l'oggetto con la mano virtuale;

- * **postcondizioni:** l'oggetto è traslato lungo un piano;
- * **scenario principale:**
 1. l'utente afferra l'oggetto con la mano virtuale;
 2. l'utente spostando la mano nello spazio fa muovere l'oggetto lungo un piano.

4.2.7. UC2.2 - Traslazione di un oggetto lungo un piano avendolo afferrato con il controller

- * **diagramma di riferimento:** *figura 9*;
- * **attore principale:** utente;
- * **precondizioni:** l'utente afferra l'oggetto con il controller;
- * **postcondizioni:** l'oggetto è traslato lungo un piano;
- * **scenario principale:**
 1. l'utente afferra l'oggetto premendo il pulsante inferiore del controller;
 2. l'utente spostando la mano che impugna il controller nello spazio fa muovere l'oggetto lungo un piano.

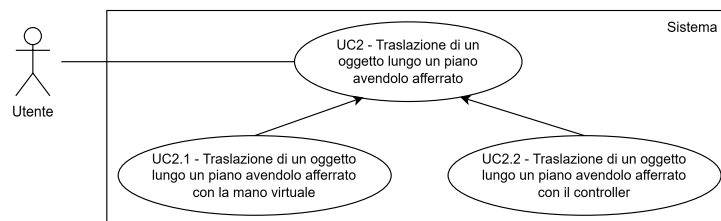


Figura 9: Diagramma dei casi d'uso UC2, UC2.1 e UC2.2

4.2.8. UC3 - Rotazione di un oggetto lungo un asse avendolo afferrato

- * **diagramma di riferimento:** *figura 10*;
- * **attore principale:** utente;
- * **precondizioni:** l'utente afferra l'oggetto;
- * **postcondizioni:** l'oggetto è ruotato lungo un asse;
- * **scenario principale:**
 1. l'utente afferra l'oggetto;
 2. l'utente inclinando la mano nello spazio fa ruotare l'oggetto lungo un asse;
- * **generalizzazioni:**
 1. Rotazione di un oggetto lungo un asse avendolo afferrato con la mano virtuale (*UC3.1*);
 2. Rotazione di un oggetto lungo un asse avendolo afferrato con il controller (*UC3.2*).

4.2.9. UC3.1 - Rotazione di un oggetto lungo un asse avendolo afferrato con la mano virtuale

- * **diagramma di riferimento:** *figura 10*;
- * **attore principale:** utente;

- * **precondizioni:** l'utente afferra l'oggetto con la mano virtuale;
- * **postcondizioni:** l'oggetto è ruotato lungo un asse;
- * **scenario principale:**
 1. l'utente afferra l'oggetto con la mano virtuale;
 2. l'utente inclinando la mano nello spazio fa ruotare l'oggetto lungo un asse.

4.2.10. UC3.2 - Rotazione di un oggetto lungo un asse avendolo afferrato con il controller

- * **diagramma di riferimento:** *figura 10*;
- * **attore principale:** utente;
- * **precondizioni:** l'utente afferra l'oggetto con il controller;
- * **postcondizioni:** l'oggetto è ruotato lungo un asse;
- * **scenario principale:**
 1. l'utente afferra l'oggetto premendo il pulsante inferiore del controller;
 2. l'utente inclinando la mano che impugna il controller nello spazio fa ruotare l'oggetto lungo un asse.

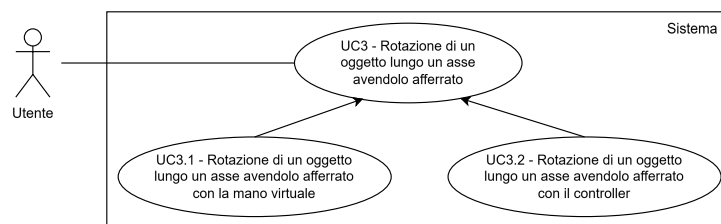


Figura 10: Diagramma dei casi d'uso UC3, UC3.1 e UC3.2

4.2.11. UC4 - Ancoraggio automatico di un oggetto in un preciso alloggio

- * **diagramma di riferimento:** *figura 11*;
- * **attore principale:** utente;
- * **precondizioni:** l'utente afferra l'oggetto e lo avvicina all'alloggio;
- * **postcondizioni:** l'oggetto si posiziona nell'alloggio rimanendo afferrato dall'utente;
- * **scenario principale:**
 1. l'utente avvicina l'oggetto afferrato all'alloggio;
 2. l'oggetto viene portato nella posizione dell'alloggio e con una rotazione precisa;
 3. l'utente non perde la presa dell'oggetto;
- * **generalizzazioni:**
 1. Ancoraggio automatico di un oggetto in un preciso alloggio afferrato con la mano virtuale (*UC4.1*);
 2. Ancoraggio automatico di un oggetto in un preciso alloggio afferrato con il controller (*UC4.2*).

4.2.12. UC4.1 - Ancoraggio automatico di un oggetto in un preciso alloggio afferrato con la mano virtuale

- * **diagramma di riferimento:** *figura 11*;
- * **attore principale:** utente;
- * **precondizioni:** l'utente afferra l'oggetto con la mano virtuale e lo avvicina all'alloggio;
- * **postcondizioni:** l'oggetto si posiziona nell'alloggio rimanendo afferrato dall'utente;
- * **scenario principale:**
 1. l'utente avvicina l'oggetto afferrato con la mano virtuale all'alloggio;
 2. l'oggetto viene portato nella posizione dell'alloggio e con una rotazione precisa;
 3. l'utente non perde la presa dell'oggetto con la mano virtuale.

4.2.13. UC4.2 - Ancoraggio automatico di un oggetto in un preciso alloggio afferrato con il controller

- * **diagramma di riferimento:** *figura 11*;
- * **attore principale:** utente;
- * **precondizioni:** l'utente afferra l'oggetto con il controller e lo avvicina all'alloggio;
- * **postcondizioni:** l'oggetto si posiziona nell'alloggio rimanendo afferrato dall'utente;
- * **scenario principale:**
 1. l'utente afferra l'oggetto premendo il pulsante inferiore del controller;
 2. l'utente avvicina l'oggetto afferrato all'alloggio;
 3. l'oggetto viene portato nella posizione dell'alloggio e con una rotazione precisa;
 4. l'utente non perde la presa dell'oggetto con il controller.

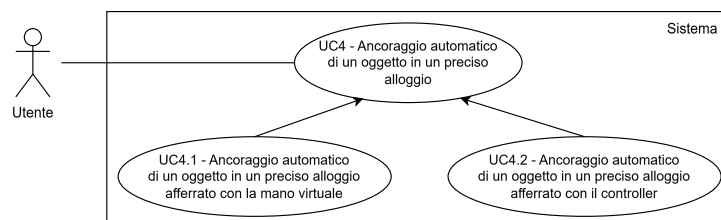


Figura 11: Diagramma dei casi d'uso UC4, UC4.1 e UC4.2

4.2.14. UC5 - Interazione con una interfaccia utente

- * **diagramma di riferimento:** *figura 12*;
- * **attore principale:** utente;
- * **precondizioni:** l'utente visualizza una interfaccia utente nello spazio virtuale;
- * **postcondizioni:** l'elemento dell'interfaccia è selezionato;
- * **scenario principale:**
 1. l'utente punta l'elemento dell'interfaccia utente;
 2. l'utente interagisce con l'elemento;
 3. l'elemento viene selezionato;
- * **generalizzazioni:**
 1. Interazione con una interfaccia utente utilizzando la mano virtuale (*UC5.1*);

2. Interazione con una interfaccia utente utilizzando il controller (*UC5.2*).

4.2.15. UC5.1 - Interazione con una interfaccia utente utilizzando la mano virtuale

- * **diagramma di riferimento:** *figura 12*;
- * **attore principale:** utente;
- * **precondizioni:** l'utente visualizza una interfaccia utente nello spazio virtuale;
- * **postcondizioni:** l'elemento dell'interfaccia è selezionato;
- * **scenario principale:**
 1. l'utente punta l'elemento dell'interfaccia utente utilizzando la mano virtuale;
 2. l'utente interagisce con l'elemento unendo pollice e indice;
 3. l'elemento viene selezionato.

4.2.16. UC5.2 - Interazione con una interfaccia utente utilizzando il controller

- * **diagramma di riferimento:** *figura 12*;
- * **attore principale:** utente;
- * **precondizioni:** l'utente visualizza una interfaccia utente nello spazio virtuale;
- * **postcondizioni:** l'elemento dell'interfaccia è selezionato;
- * **scenario principale:**
 1. l'utente punta l'elemento dell'interfaccia utente utilizzando il controller;
 2. l'utente interagisce con l'elemento premendo il pulsante frontale del controller;
 3. l'elemento viene selezionato.

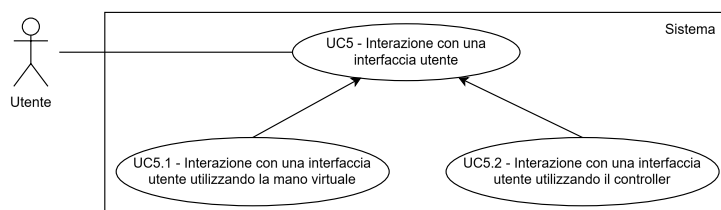


Figura 12: Diagramma dei casi d'uso UC5, UC5.1 e UC5.2

4.2.17. UC6 - Visualizzazione dati aziendali su interfaccia utente

- * **diagramma di riferimento:** *figura 13*;
- * **attore principale:** utente;
- * **precondizioni:** l'utente avvia l'applicazione;
- * **postcondizioni:** l'utente visualizza alcuni dati dell'azienda che detiene l'applicazione su una interfaccia;
- * **scenario principale:**
 1. l'applicazione viene avviata correttamente;
 2. l'utente visualizza alcuni dati dell'azienda su un'interfaccia utente;
- * **inclusioni:**

1. Visualizzazione nome dell'azienda su interfaccia utente (UC6.1);
2. Visualizzazione logo dell'azienda su interfaccia utente (UC6.2).

4.2.18. UC6.1 - Visualizzazione nome dell'azienda su interfaccia utente

- * **diagramma di riferimento:** *figura 13*;
- * **attore principale:** utente;
- * **precondizioni:** l'utente avvia l'applicazione;
- * **postcondizioni:** l'utente visualizza il nome dell'azienda su una interfaccia;
- * **scenario principale:**
 1. l'applicazione viene avviata correttamente;
 2. l'utente visualizza il nome dell'azienda.

4.2.19. UC6.2 - Visualizzazione logo dell'azienda su interfaccia utente

- * **diagramma di riferimento:** *figura 13*;
- * **attore principale:** utente;
- * **precondizioni:** l'utente avvia l'applicazione;
- * **postcondizioni:** l'utente visualizza il logo dell'azienda su una interfaccia;
- * **scenario principale:**
 1. l'applicazione viene avviata correttamente;
 2. l'utente visualizza il logo dell'azienda.

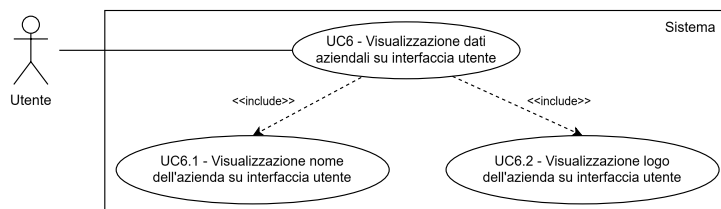


Figura 13: Diagramma dei casi d'uso UC6, UC6.1 e UC6.2

4.2.20. UC7 - Visualizzazione menu principale

- * **diagramma di riferimento:** *figura 14*;
- * **attore principale:** utente;
- * **precondizioni:** l'utente avvia correttamente l'applicazione;
- * **postcondizioni:** il menu è esposto di fronte all'utente;
- * **scenario principale:**
 1. l'applicazione viene avviata correttamente;
 2. l'utente visualizza il menu principale;
- * **inclusioni:**
 1. Visualizzazione lista delle procedure nel menu principale (UC7.1).

4.2.21. UC7.1 - Visualizzazione lista delle procedure nel menu principale

- * **diagramma di riferimento:** *figura 14*;
- * **attore principale:** utente;
- * **precondizioni:** l'utente visualizza il menu principale;
- * **postcondizioni:** viene esposta la lista delle procedure;
- * **scenario principale:**
 1. l'applicazione viene avviata correttamente;
 2. l'utente visualizza il menu principale con la lista delle procedure da poter scegliere;
- * **inclusioni:**
 1. Visualizzazione singola procedura nella lista del menu principale (*UC7.1.1*).

4.2.22. UC7.1.1 - Visualizzazione singola procedura nella lista del menu principale

- * **diagramma di riferimento:** *figura 14*;
- * **attore principale:** utente;
- * **precondizioni:** l'utente visualizza la lista delle procedure nel menu principale;
- * **postcondizioni:** viene esposto l'elemento della lista;
- * **scenario principale:**
 1. l'applicazione viene avviata correttamente;
 2. l'utente visualizza l'elemento della lista delle procedure;
- * **inclusioni:**
 1. Visualizzazione nome della procedura nell'elemento della lista (*UC7.1.1.1*).

4.2.23. UC7.1.1.1 - Visualizzazione nome della procedura nell'elemento della lista

- * **diagramma di riferimento:** *figura 14*;
- * **attore principale:** utente;
- * **precondizioni:** l'utente visualizza l'elemento nella lista delle procedure;
- * **postcondizioni:** viene esposto il nome della procedura;
- * **scenario principale:**
 1. l'utente visualizza l'elemento della lista delle procedure;
 2. l'utente visualizza il nome della procedura all'interno dell'elemento.

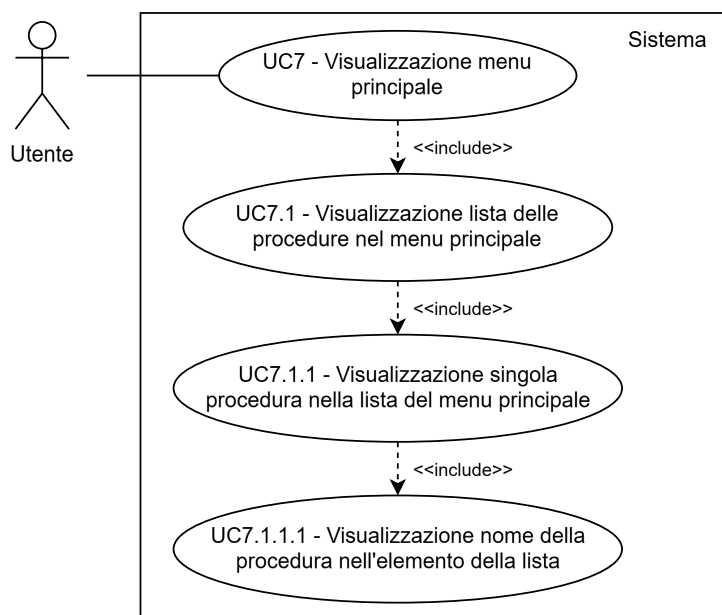


Figura 14: Diagramma dei casi d'uso UC7, UC7.1, UC7.1.1 e UC7.1.1.1

4.2.24. UC8 - Visualizzazione dettagli procedura selezionata

- * **diagramma di riferimento:** *figura 15*;
- * **attore principale:** utente;
- * **precondizioni:** l'utente seleziona una procedura dalla lista del menu principale;
- * **postcondizioni:** vengono esposti i dettagli della procedura;
- * **scenario principale:**
 1. l'utente seleziona una procedura dalla lista del menu principale;
 2. l'utente visualizza i dettagli della procedura;
- * **inclusioni:**
 1. Visualizzazione nome della procedura nei dettagli (*UC8.1*);
 2. Visualizzazione descrizione della procedura nei dettagli (*UC8.2*);
 3. Visualizzazione lista degli step nei dettagli della procedura (*UC8.3*).

4.2.25. UC8.1 - Visualizzazione nome della procedura nei dettagli

- * **diagramma di riferimento:** *figura 15*;
- * **attore principale:** utente;
- * **precondizioni:** l'utente visualizza i dettagli della procedura selezionata;
- * **postcondizioni:** viene esposto il nome della procedura;
- * **scenario principale:**
 1. l'utente seleziona una procedura dalla lista del menu principale;
 2. l'utente visualizza il nome della procedura.

4.2.26. UC8.2 - Visualizzazione descrizione della procedura nei dettagli

- * **diagramma di riferimento:** *figura 15*;
- * **attore principale:** utente;
- * **precondizioni:** l'utente visualizza i dettagli della procedura selezionata;
- * **postcondizioni:** viene esposto il descrizione della procedura;
- * **scenario principale:**
 1. l'utente seleziona una procedura dalla lista del menu principale;
 2. l'utente visualizza il descrizione della procedura.

4.2.27. UC8.3 - Visualizzazione lista degli step nei dettagli della procedura

- * **diagramma di riferimento:** *figura 15*;
- * **attore principale:** utente;
- * **precondizioni:** l'utente visualizza i dettagli della procedura selezionata;
- * **postcondizioni:** viene esposta la lista degli step;
- * **scenario principale:**
 1. l'utente seleziona una procedura dalla lista del menu principale;
 2. l'utente visualizza la lista degli step della procedura;
- * **inclusioni:**
 1. Visualizzazione singolo step nella lista dei dettagli della procedura (*UC8.3.1*).

4.2.28. UC8.3.1 - Visualizzazione singolo step nella lista dei dettagli della procedura

- * **diagramma di riferimento:** *figura 16*;
- * **attore principale:** utente;
- * **precondizioni:** l'utente visualizza la lista degli step nei dettagli della procedura;
- * **postcondizioni:** viene esposto l'elemento della lista;
- * **scenario principale:**
 1. l'utente seleziona una procedura dalla lista del menu principale;
 2. l'utente visualizza l'elemento della lista degli step;
- * **inclusioni:**
 1. Visualizzazione nome dello step nell'elemento della lista (*UC8.3.1.1*).

4.2.29. UC8.3.1.1 - Visualizzazione nome dello step nell'elemento della lista

- * **diagramma di riferimento:** *figura 16*;
- * **attore principale:** utente;
- * **precondizioni:** l'utente visualizza l'elemento nella lista degli step;
- * **postcondizioni:** viene esposto il nome dello step;

* **scenario principale:**

1. l'utente visualizza l'elemento della lista degli step;
2. l'utente visualizza il nome dello step all'interno dell'elemento.

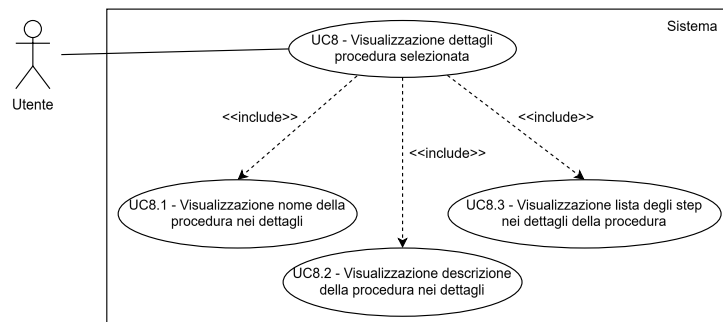


Figura 15: Diagramma dei casi d'uso UC8, UC8.1, UC8.2 e UC8.3

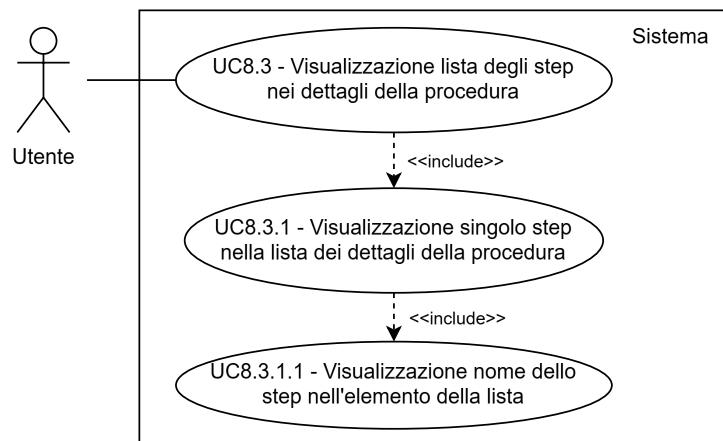


Figura 16: Diagramma dei casi d'uso UC8.3, UC8.3.1 e UC8.3.1.1

4.2.30. UC9 - Avvio della procedura selezionata

* **diagramma di riferimento:** *figura 17*;

* **attore principale:** utente;

* **precondizioni:** l'utente visualizza i dettagli della procedura selezionata;

* **postcondizioni:** la procedura è avviata;

* **scenario principale:**

1. l'utente visualizza i dettagli della procedura selezionata;
2. l'utente interagisce con l'interfaccia per avviare la procedura;
3. la procedura viene avviata.

4.2.31. UC10 - Visualizzazione stato di avanzamento della procedura

* **diagramma di riferimento:** *figura 17*;

* **attore principale:** utente;

* **precondizioni:** l'utente avvia una procedura;

- * **postcondizioni:** l'utente visualizza lo stato di avanzamento della procedura in esecuzione;
- * **scenario principale:**
 1. l'utente avvia una procedura;
 2. viene esposta una interfaccia con il numero dello step corrente sul totale e la descrizione per completarlo.

4.2.32. UC11 - Riavvio della procedura in esecuzione

- * **diagramma di riferimento:** *figura 17*;
- * **attore principale:** utente;
- * **precondizioni:** una procedura è in esecuzione;
- * **postcondizioni:** la procedura è stata riavviata;
- * **scenario principale:**
 1. la procedura si trova a un certo stato di avanzamento;
 2. l'utente interagisce con l'interfaccia per riavviare la procedura;
 3. viene caricato il primo step della procedura e vengono persi gli avanzamenti precedenti.

4.2.33. UC12 - Interruzione della procedura in esecuzione

- * **diagramma di riferimento:** *figura 17*;
- * **attore principale:** utente;
- * **precondizioni:** l'utente visualizza un elemento sull'interfaccia per interrompere la procedura;
- * **postcondizioni:** la procedura è interrotta;
- * **scenario principale:**
 1. l'utente visualizza un elemento sull'interfaccia per interrompere la procedura;
 2. l'utente interagisce con l'interfaccia per interrompere la procedura in esecuzione;
 3. la procedura viene interrotta;
 4. Viene ricaricato il menu principale.

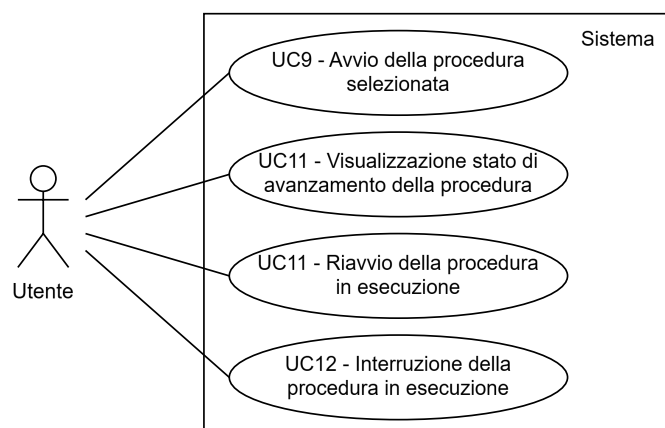


Figura 17: Diagramma dei casi d'uso UC9, UC10, UC11 e UC12

4.2.34. UC13 - Avvio dello step selezionato

- * **diagramma di riferimento:** *figura 18*;
- * **attore principale:** utente;
- * **precondizioni:** l'utente visualizza l'elemento della lista degli step;
- * **postcondizioni:** lo step è avviato;
- * **scenario principale:**
 1. l'utente visualizza uno step della procedura selezionata;
 2. l'utente interagisce con l'interfaccia per avviare lo step;
 3. lo step viene avviato;
- * **inclusioni:**
 1. Utilizzo esclusivo degli elementi dello step in esecuzione (*UC13.1*);
 2. Evidenziazione degli elementi dello step in esecuzione (*UC13.2*).

4.2.35. UC13.1 - Utilizzo esclusivo degli elementi dello step in esecuzione

- * **diagramma di riferimento:** *figura 18*;
- * **attore principale:** utente;
- * **precondizioni:** uno step è stato caricato correttamente;
- * **postcondizioni:** l'utente interagisce con gli elementi dello step;
- * **scenario principale:**
 1. lo step è stato avviato correttamente;
 2. l'utente interagisce unicamente con gli elementi necessari al completamento dello step.

4.2.36. UC13.2 - Evidenziazione degli elementi dello step in esecuzione

- * **diagramma di riferimento:** *figura 18*;
- * **attore principale:** utente;
- * **precondizioni:** uno step è stato caricato correttamente;
- * **postcondizioni:** vengono evidenziati gli elementi necessari al completamento dello step;
- * **scenario principale:**
 1. lo step è stato avviato correttamente;
 2. l'utente visualizza gli elementi necessari al completamento dello step evidenziati.

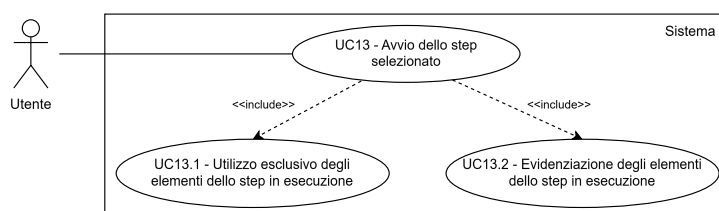


Figura 18: Diagramma dei casi d'uso UC13, UC13.1 e UC13.2

4.2.37. UC14 - Visualizzazione dettagli step in esecuzione

- * **diagramma di riferimento:** *figura 19*;
- * **attore principale:** utente;
- * **precondizioni:** lo step viene avviato correttamente;
- * **postcondizioni:** vengono esposti i dettagli dello step;
- * **scenario principale:**
 1. lo step viene avviato come primo della procedura oppure dopo la selezione dell'utente;
 2. l'utente visualizza i dettagli dello step;
- * **inclusioni:**
 1. Visualizzazione nome dello step nei dettagli (*UC14.1*);
 2. Visualizzazione descrizione dello step nei dettagli (*UC14.2*).

4.2.38. UC14.1 - Visualizzazione nome dello step nei dettagli

- * **diagramma di riferimento:** *figura 19*;
- * **attore principale:** utente;
- * **precondizioni:** l'utente visualizza i dettagli dello step selezionato;
- * **postcondizioni:** viene esposto il nome dello step;
- * **scenario principale:**
 1. lo step viene avviato come primo della procedura oppure dopo la selezione dell'utente;
 2. l'utente visualizza il nome dello step.

4.2.39. UC14.2 - Visualizzazione descrizione dello step nei dettagli

- * **diagramma di riferimento:** *figura 19*;
- * **attore principale:** utente;
- * **precondizioni:** l'utente visualizza i dettagli dello step selezionato;
- * **postcondizioni:** viene esposta la descrizione dello step;
- * **scenario principale:**
 1. lo step viene avviato come primo della procedura oppure dopo la selezione dell'utente;
 2. l'utente visualizza la descrizione dello step.

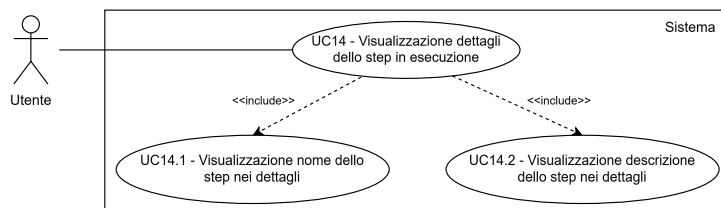


Figura 19: Diagramma dei casi d'uso UC14, UC14.1 e UC14.2

4.2.40. UC15 - Riavvio dello step in esecuzione

- * **diagramma di riferimento:** *figura 20*;
- * **attore principale:** utente;
- * **precondizioni:** uno step è in esecuzione;
- * **postcondizioni:** lo step è stato riavviato;
- * **scenario principale:**
 1. lo step si trova a un certo stato di avanzamento;
 2. l'utente interagisce con l'interfaccia per riavviare lo step.

4.2.41. UC16 - Interruzione dello step in esecuzione

- * **diagramma di riferimento:** *figura 20*;
- * **attore principale:** utente;
- * **precondizioni:** l'utente visualizza un elemento sull'interfaccia per interrompere lo step;
- * **postcondizioni:** la step è interrotto;
- * **scenario principale:**
 1. l'utente visualizza un elemento sull'interfaccia per interrompere lo step;
 2. l'utente interagisce con l'interfaccia per interrompere lo step in esecuzione;
 3. lo step viene interrotto;
 4. viene ricaricato il pannello con i dettagli della procedura e la lista degli step.

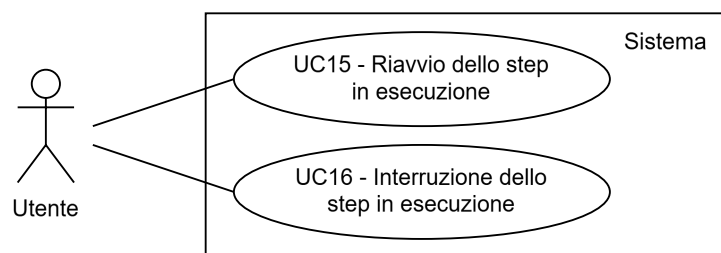


Figura 20: Diagramma dei casi d'uso UC15 e UC16

4.3. Tracciamento dei requisiti

Ogni requisito derivato dall'analisi svolta in precedenza viene tracciato nelle tabelle sotto-

stanti:

- * Requisiti funzionali obbligatori (*tabella 1*);
- * Requisiti funzionali desiderabili (*tabella 2*);
- * Requisiti funzionali facoltativi (*tabella 3*);
- * Requisiti di qualità obbligatori (*tabella 4*);
- * Requisiti di vincolo obbligatori (*tabella 5*);
- * Requisiti di vincolo desiderabili (*tabella 6*).

Molti di essi sono generati dai casi d'uso, alcuni invece provengono da riflessioni con il tutor aziendale.

Ogni requisito è identificato da un codice univoco strutturato nella seguente maniera:

R[Tipologia][Priorità]-[Identificativo univoco]

Dove:

* la tipologia si distingue in tre casi:

- **F** - requisito funzionale;
- **Q** - requisito di qualità;
- **V** - requisito di vincolo.

* la priorità è divisa in tre livelli:

- **O** - obbligatorio, necessario affinché l'applicazione possa definirsi completata positivamente;
- **D** - desiderabile, non necessario al completamento dell'applicazione ma attribuisce un significativo valore aggiunto;
- **F** - facoltativo, non necessario al completamento dell'applicazione, apporta benefici limitati.

* il codice univoco è un numero intero positivo.

4.3.1. Requisiti funzionali

4.3.1.1. Requisiti obbligatori

Tabella 1: Tracciamento dei requisiti funzionali obbligatori

Codice	Descrizione	Fonti
RFO-1	L'utente deve potersi muovere nello spazio virtuale utilizzando i controller.	<u>UC1</u> <u>UC1.1</u> <u>UC1.2</u> <u>UC1.3</u>
RFO-2	L'utente deve poter afferrare un oggetto e traslarlo lungo un piano.	<u>UC2</u> <u>UC2.1</u> <u>UC2.2</u>
RFO-3	L'utente deve poter afferrare un oggetto e ruotarlo lungo un asse.	<u>UC3</u> <u>UC3.1</u> <u>UC3.2</u>
RFO-4	L'utente deve poter afferrare un oggetto e ancorarlo in un alloggio preciso. L'oggetto deve rimanere nella presa dell'utente e subire i vincoli di movimento dell'alloggio.	<u>UC4</u> <u>UC4.1</u> <u>UC4.2</u>
RFO-5	L'utente deve poter interagire con una interfaccia utente.	<u>UC5</u> <u>UC5.1</u> <u>UC5.2</u>
RFO-6	Ogni interazione con gli oggetti o le interfacce utente deve poter essere svolta con le mani virtuali.	<u>UC2.1</u> <u>UC3.1</u> <u>UC4.1</u> <u>UC5.1</u>

Codice	Descrizione	Fonti
RFO-7	Ogni interazione con gli oggetti o le interfacce utente deve poter essere svolta con i controller.	<u>UC2.2</u> <u>UC3.2</u> <u>UC4.2</u> <u>UC5.2</u>
RFO-8	L'utente deve poter visualizzare un menu principale con la lista delle procedure.	<u>UC7</u> <u>UC7.1</u> <u>UC7.1.1</u> <u>UC7.1.1.1</u>
RFO-9	L'utente deve poter selezionare e avviare una qualsiasi procedura.	<u>UC9</u>
RFO-10	L'utente deve poter visualizzare lo stato di avanzamento della procedura, quindi il numero di step superati sul totale.	<u>UC10</u>
RFO-11	L'utente deve poter riavviare la procedura in esecuzione.	<u>UC11</u>
RFO-12	L'utente deve poter interrompere la procedura in esecuzione in modo da poterne scegliere un'altra.	<u>UC12</u>
RFO-13	L'utente deve poter visualizzare il nome dello step in esecuzione e la descrizione per completarlo.	<u>UC14</u> <u>UC14.1</u> <u>UC14.2</u>

4.3.1.2. Requisiti desiderabili

Tabella 2: Tracciamento dei requisiti funzionali desiderabili

Codice	Descrizione	Fonti
RFD-1	L'utente deve poter visualizzare il nome e la descrizione della procedura.	<u>UC8</u> <u>UC8.1</u> <u>UC8.2</u>
RFD-2	L'utente deve poter visualizzare la lista degli step della procedura selezionata.	<u>UC8.3</u> <u>UC8.3.1</u> <u>UC8.3.1.1</u>
RFD-3	L'utente deve poter selezionare e avviare un qualsiasi step all'interno della procedura selezionata.	<u>UC13</u>
RFD-4	L'utente deve poter riavviare lo step in esecuzione.	<u>UC15</u>
RFD-5	L'utente deve poter interrompere lo step in esecuzione in modo da poterne scegliere un altro.	<u>UC16</u>
RFD-6	Una volta avviato uno step l'utente deve poter interagire unicamente con gli elementi necessari al completamento dello stesso.	<u>UC13.1</u>
RFD-7	Una volta avviato uno step si devono evidenziare gli elementi necessari al completamento dello stesso.	<u>UC13.2</u>

4.3.1.3. Requisiti facoltativi

Tabella 3: Tracciamento dei requisiti funzionali facoltativi

Codice	Descrizione	Fonti
RFF-1	L'utente deve poter visualizzare il nome e il logo dell'azienda detentrica dell'applicazione.	<u>UC6</u> <u>UC6.1</u> <u>UC6.2</u>

4.3.2. Requisiti di qualità

4.3.2.1. Requisiti obbligatori

Tabella 4: Tracciamento dei requisiti di qualità obbligatori

Codice	Descrizione	Fonti
RQO-1	L'applicazione deve utilizzare l'interfaccia API OpenXR rispetto a quella Oculus per permettere una portabilità maggiore sui dispositivi.	Confronto con l'azienda

4.3.3. Requisiti di vincolo

4.3.3.1. Requisiti obbligatori

Tabella 5: Tracciamento dei requisiti di vincolo obbligatori

Codice	Descrizione	Fonti
RVO-1	L'applicazione deve essere disponibile su piattaforma Android, sistema operativo dei visori Meta Quest.	Piano di Lavoro Confronto con l'azienda
RVO-2	L'applicazione deve essere disponibile su dispositivo standalone, installabile e utilizzabile quindi direttamente sul visore senza l'ausilio di un computer.	Piano di Lavoro Confronto con l'azienda

4.3.3.2. Requisiti desiderabili

Tabella 6: Tracciamento dei requisiti di vincolo desiderabili

Codice	Descrizione	Fonti
RVD-1	Al completamento di uno step deve essere caricato automaticamente il successivo. In caso sia l'ultimo step della procedura questa deve essere conclusa sempre automaticamente.	<u>Diagramma di sequenza</u>

Capitolo 5

Progettazione

Nella seguente sezione vengono espone le scelte architetture e strutturali adottate per l'applicazione. A fini didattici si è voluto aggiungere inoltre una componente di backend, seppur semplice, per dimostrare lo scambio di dati tra l'applicazione e un sistema di base di dati.

5.1. Architettura della scena

Per una migliore struttura della scena in Unity si sono suddivisi i GameObject in quattro categorie: manager, scene, props¹ e *User Interface (UI)*. Nei primi rientrano gli oggetti che hanno il compito di gestire alcune funzionalità, in genere infatti sono singleton; i secondi descrivono gli aspetti della scena come le luci e i suoni ambientali; i props sono gli oggetti effettivi della scena con i quali interagire; gli ultimi sono le interfacce grafiche immerse nella scena virtuale. Nelle seguenti sezioni sono analizzate le scelte architetture degli elementi di maggiore rilievo, vale a dire alcuni manager e il Camera Rig.

5.1.1. Ordine di esecuzione dei manager

Unity esegue gli script legati agli oggetti nell'ordine in cui questi appaiono nella gerarchia: un oggetto che è posizionato più in alto nel pannello della hierarchy di Unity viene eseguito prima di un oggetto che gli sta sotto. Questo può comportare a dei riferimenti nulli perché la classe referenziata può non essere ancora stata creata. Nelle impostazioni si può indicare uno specifico ordine di creazione delle classi, tuttavia per le esigenze del progetto è stato sufficiente posizionare nella maniera corretta i seguenti manager:

- * **App Installer** - sfrutta la libreria Zenject per risolvere la dependency injection di alcune classi. Poiché risolve le dipendenze affinché le classi possano lavorare deve essere la prima a essere istanziata. Il suo funzionamento viene descritto in dettaglio nella *sezione 5.5.1.1*;
- * **Main Controller** - gestisce il data context² dei dati della compagnia da recuperare dal sistema di base di dati. Anch'esso dovrebbe essere il primo ad essere eseguito, ma le sue dipendenze vengono risolte da AppInstaller quindi viene posizionato immediatamente sotto nella gerarchia. Il suo funzionamento viene descritto in dettaglio nella *sezione 5.5.5*;
- * **MonoBehaviour Reference** - alcune classi non legate a GameObject hanno necessitato l'uso di un meccanismo di threading gestito dalla classe MonoBehaviour di Unity. Non è possibile estenderle alla MonoBehaviour poiché questa rimuove i costruttori demandando l'istanziamento a Unity stesso. La soluzione adottata è stata quindi quella di creare un singleton che potesse essere richiamato in modo da sfruttare la gestione dei thread.

5.1.2. Struttura del Camera Rig

Il Camera Rig è il GameObject che consente all'utente di visualizzare la scena virtuale, le mani e i controller, di potersi muovere e interagire con l'ambiente. Questo è stato composto

¹Dall'inglese, oggetti che allestiscono una scenografia.

²Stato di un contesto, ad esempio condizione di alcuni dati dell'applicazione.

partendo dal building block offerto dalla SDK e integrato con componenti personalizzati e prefab. La versione ultima utilizzata nel progetto è strutturata nel seguente modo:

- * **tracking space**

- ▶ **eyes tracking** - componenti per la gestione delle camera del visore;
- ▶ **hand anchors** - punti di ancoraggio delle mani per tracciare i movimenti e i modelli 3D delle mani virtuali e dei controller;

- * **interaction**

- ▶ **OVRHmd** - gestore dei dati riguardanti lo scheletro delle mani;
- ▶ **hand interactions** - gestore dei dati riguardanti le interazioni delle mani virtuali, contiene gli interactor e i modelli 3D;
- ▶ **controller interactions** - gestore dei dati riguardanti le interazioni dei joypad, contiene gli interactor e i modelli 3D;
- ▶ **locomotor** - gestore dei movimenti nello spazio del personaggio e dei rapporti di collisione e fisica.

5.2. Architettura delle procedure

La struttura più complessa dell'applicazione è quella che definisce le procedure da eseguire sul macchinario. Sono stati dedicati alcuni giorni di studio teorico in modo tale da rendere più efficiente il processo di progettazione e implementazione.

5.2.1. Definizione con un automa a stati finiti non deterministici

Le procedure sono descrivibili come degli automi a stati finiti non deterministici dove gli stati sono gli step. Si passa allo stato successivo quando lo step corrente è considerato completato ad eccezione dell'ultimo che fa terminare la procedura. Gli step possono essere riutilizzati in più procedimenti, ad esempio l'apertura delle porte del macchinario può essere un passaggio comune a più procedure. Dati quindi gli step $S = \{s_i : i \in \mathbb{N} \setminus \{0\}\}$ si può quindi definire il seguente automa:

$$A_P = (Q, \Sigma, \delta, q_0, F)$$

Dove:

- * $Q = \{q_i : i \in \{0, \dots, |S|\}\}$
insieme di stati;
- * $\Sigma = S \cup \{\varepsilon\}$
alfabeto;
- * $\delta = Q \times \Sigma \rightarrow Q \Rightarrow \delta(q, a) = q_i$ dove $i \in \{1, \dots, |S|\}$ se $q = q_0, a = \varepsilon$ oppure $q \neq q_0, a \neq \varepsilon$
funzione di transizione;
- * $q_0 \in Q$
stato iniziale;
- * $F = Q \setminus \{q_0\}$
stati finali.

Il linguaggio L_P quindi è l'insieme delle procedure che si possono formare dalla combinazione degli step S . Un esempio dell'automa con tre step è rappresentato in [figura 21](#), mentre una possibile procedura p è la seguente:

$$p = s2s1s3$$

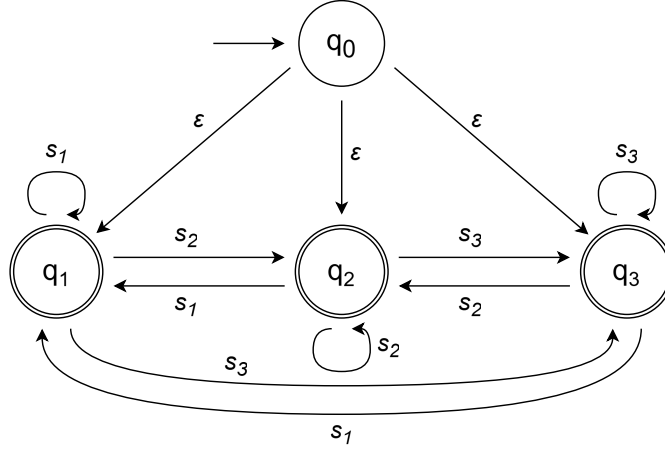


Figura 21: Automa che accetta tutte le procedure componibili con tre step

L'automa definisce in un certo senso anche le azioni che può compiere l'utente. Questo infatti può scegliere di eseguire l'intera procedura p o solo un suo step, quindi una sottostringa di p di lunghezza uno; gli è concesso di interrompere la procedura o lo step durante l'esecuzione ma lo manterrebbe sempre in uno stato finale, che quindi non porta a errori; infine può riavviare la procedura o lo step, azione possibile in quanto ogni q_i è sempre raggiungibile da q_j dove $i, j \in \{1, \dots, |S|\}$. Un esempio di azione u dell'utente sulla procedura p può essere quindi la seguente:

$$u = s2s1s2s1s3$$

L'utente ha avviato la procedura, eseguiti i primi due step, poi ha fatto ripartire la procedura riuscendo a completarla.

5.2.2. Struttura delle procedure

Ogni step contiene i riferimenti agli elementi necessari al completamento dello stesso, i loro stati iniziali e la condizione per la quale si può considerare lo step compiuto. Un elemento può essere condiviso tra più step e per questo può possedere diversi stati iniziali. Lo step associa quindi ad ogni proprio elemento un inizializzatore che ne definisce lo stato iniziale. Un esempio è fornito in [figura 22](#) nel quale viene richiesto la posizione e la rotazione iniziale, e se l'oggetto è afferrabile o meno. I campi sono preceduti dall'attributo [SerializeField] così da permettere l'assegnazione via Editor. L'elemento che riceverà questo tipo di inizializzatore potrà accedere ai campi in sola lettura utilizzando le proprietà.

```
3 public class GrabTransformerStepElementInitializer : MonoBehaviour, IStepElementInitializer
4 {
5     [SerializeField]
6     private Vector3 _localPosition;
7     [SerializeField]
8     private Vector3 _localRotation;
9     [SerializeField]
10    private bool _isGrabbable;
11
12    3 references
13    public Vector3 LocalPosition => _localPosition;
14    2 references
15    public Quaternion LocalRotation => Quaternion.Euler(_localRotation);
16    5 references
17    public bool IsGrabbable => _isGrabbable;
18 }
```

Figura 22: Esempio di un iniziatore di un elemento

Un manager tiene quindi traccia di tutte le procedure disponibili, queste contengono gli step in ordine di esecuzione i quali a loro volta possiedono i riferimenti agli elementi e i relativi iniziatore. Alla selezione di una procedura vengono effettuate le seguenti chiamate a cascata:

* inizializzazione:

- ▶ il manager chiede alla procedura di inicializzarsi;
- ▶ la procedura chiede ai propri step di inicializzarsi. Questa operazione viene svolta dall'ultimo step al primo cosicchè se un elemento è condiviso in più passaggi il suo stato iniziale viene sovrascritto dallo step che si incontrerà prima. In questo modo l'elemento non verrà visualizzato in uno stato intermedio della procedura;
- ▶ gli step chiedono ai propri elementi di inicializzarsi passandogli i relativi iniziatore;
- ▶ gli elementi memorizzano lo stato iniziale ricevuto dallo step;

* pulizia:

- ▶ la procedura chiede agli step di ripulire³ la scena. La richiesta avviene in ordine inverso di esecuzione per lo stesso motivo dell'inizializzazione;
- ▶ gli step chiedono ai propri elementi di ripulirsi;
- ▶ gli elementi assumo lo stato iniziale ma sono resi non interagibili;

* avvio:

- ▶ la procedura chiede al primo step di avviarsi;
- ▶ lo step chiede ai propri elementi di avviarsi;
- ▶ gli elementi assumono lo stato iniziale e vengono resi interagibili.

Questa sequenza di chiamate viene eseguita anche quando l'utente seleziona un singolo step all'interno di una procedura, ma in aggiunta viene avviato nuovamente il passaggio selezionato. In questo modo si ripulisce la scena reimpostandola allo stato iniziale della procedura e poi si avviano unicamente gli elementi dello step desiderato.

Ogni step, oltre ad associare ad ogni elemento il corrispettivo iniziatore, necessita dello stato da attribuire all'elemento durante l'avvio. In fase di sviluppo si può infatti selezionare una delle seguenti opzioni:

³Nel contesto corrente si intende reipostare la scena allo stato iniziale rendendola tuttavia statica, senza permettere cioè all'utente di poterci interagire.

- * **InitializeAndStart** - inizializza e avvia l'elemento, sempre utilizzato quando l'utente seleziona un singolo step. L'elemento deve essere riportato allo stato iniziale poiché non si sa come sono terminati quelli precedenti;
- * **Enable** - attiva l'elemento, lo rende quindi interagibile indipendentemente dallo stato in cui si trova. Questa situazione è desiderabile quando si vuole mantenere la condizione dell'elemento lasciata dallo step precedente. Quando ad esempio l'utente appoggia l'oggetto, all'avvio dello step successivo è desiderato che l'oggetto in questione rimanga nella posizione lasciata, semplicemente venga reso nuovamente interagibile;
- * **Disable** - disattiva l'elemento, lo rende quindi non più interagibile indipendentemente dallo stato in cui si trova. Questa situazione è simile a quella generata dall'opzione "Enable" ma l'oggetto non fa parte inizialmente degli elementi dello step.

La configurazione di un elemento, il rispettivo iniziatore e lo stato iniziale via Editor si presenta come mostrato in *figura 23*.

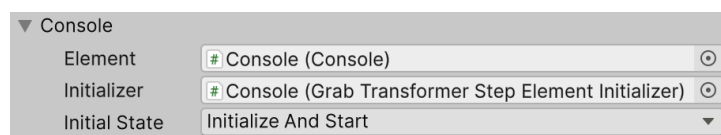


Figura 23: Configurazione di un elemento di uno step dall'Editor

Gli elementi sono infine caratterizzati da cinque stati mutuamente esclusivi, quattro dei quali sono utilizzati per definirne lo stato iniziale all'interno degli step come visto in precedenza:

- * **inizializzato** - l'elemento memorizza lo stato iniziale;
- * **avviato** - l'elemento assume lo stato iniziale memorizzato ed è interagibile;
- * **abilitato** - l'elemento è interagibile, non per forza nel suo stato iniziale;
- * **disabilitato** - l'elemento non è interagibile, non per forza nel suo stato iniziale;
- * **pulito** - l'elemento non è interagibile e viene reimpostato al suo stato iniziale. Questo viene assegnato ogni volta che lo step viene chiuso, completato o meno che sia, per ripulire la scena.

5.3. Architettura del backend

Sebbene di non fondamentale rilevanza è stato concordato con l'azienda di integrare un semplice sistema di base di dati. Questo deve essere in grado di mantenere dei dati grezzi, ad esempio stringhe e numeri, ma anche risorse come immagini. All'avvio dell'applicazione i dati e le risorse devono essere caricate e mostrate all'utente. Per l'implementazione è stato adottato il repository pattern spiegato in dettaglio nella *sezione 5.5.5*.

I dati grezzi sono memorizzati in un file .json nella cartella `StreamingAssets`. Utilizzando questa cartella si semplifica l'accesso del visore ai file in quanto è quella predefinita dal quale caricare dati. Le risorse sono salvate infatti nella stessa cartella con un percorso locale differente per mantenere una separazione tra dati e risorse. Il documento .json quindi contiene inoltre i percorsi locali per accedere alle risorse, nel caso del progetto ad una immagine .png. Prima quindi vengono caricati i dati grezzi costruendo un modello, successivamente vengono caricate le risorse componendo così un "view model" con variabili di dominio, interne cioè al sistema. Nonostante l'esiguo numero di dati si è voluto mantenere la separazione tra dati grezzi e risorse per simulare uno scenario reale. Le seconde infatti possono essere immagini, video, texture, materiali o interi

asset. L'idea sarebbe quindi quella di allocarli in remoto e mantenere in locale solo i dati per accedervi e scaricarli quando necessario, in modo da non appesantire l'applicativo.

5.4. Architettura della snap interaction

La snap interaction consiste nell'interazione di un oggetto con un suo alloggio nel quale viene posizionato e ancorato automaticamente. La SDK mette a disposizione un building block che compie questa interazione, tuttavia non è stata ritenuta funzionale: l'oggetto, tenuto in mano dall'utente, si colloca nell'alloggio solo quando viene rilasciato; una volta afferrato nuovamente torna nella presa dell'utente libero da qualunque vincolo. L'interazione desiderata invece sarebbe dovuta essere strutturata nel seguente modo: l'utente avvicina l'oggetto all'alloggio e, rimanendo nella mano dell'utente, assume delle restrizioni nel movimento e rotazione; quando vengono soddisfatte determinate condizioni, per esempio l'oggetto viene allontanato a sufficienza dall'alloggio, questo torna libero di muoversi sempre rimanendo nella presa dell'utente.

5.4.1. Progettazione e implementazione della snap interaction

La progettazione e implementazione della snap interaction è stata suddivisa in 6 passaggi sequenziali rispettando le caratteristiche precedentemente descritte. Nella seguente sezione si usa il termine “socket” per intendere l'alloggio e “interactor” per riferirsi all'oggetto:

1. **ancoraggio al socket** - affinché il passaggio fosse automatico il socket è stato dotato di un collider impostato come trigger così da rilevare le collisioni. Uno script associato al GameObject ha il compito di monitorare gli oggetti che collidono con il socket. È dotato di una lista di interactor che possono interagire con il socket, quindi controlla se l'oggetto corrisponde a un elemento della lista e in caso positivo lo ancora;
2. **mantenimento della presa dell'utente** - i componenti della SDK utilizzati per afferrare gli oggetti ne sovrascrivono la posizione in modo che segua le mani dell'utente. Per permettere di aggiornare alcune proprietà quindi è stato necessario deselezionare manualmente l'interactor, assegnare i nuovi valori e selezionarlo nuovamente alla mano. Questa operazione ha richiesto alcuni giorni di studio in quanto le classi native dalle SDK sono piuttosto corpose e scarsamente documentate;
3. **assegnazione dei vincoli** - la SDK fornisce delle classi in grado di vincolare la posizione, rotazione e dimensione dell'oggetto quando è afferrato con una o due mani. È stato pensato quindi di assegnare al GameObject dell'interactor lo script relativo ai vincoli e attivarlo solamente quando il socket lo ancora;
4. **posizionamento nel socket** - l'interactor deve allinearsi con il socket ma spesso non assumerne l'esatta posizione e rotazione, altrimenti si darebbe l'impressione che l'oggetto si teletrasporti nell'alloggio. Il socket quindi passa all'interactor la propria posizione e rotazione indicando quali componenti utilizzare. La *figura 24* mette a confronto la snap interaction con e senza override della componente orizzontale del morsetto, quella orizzontale rispetto alla visuale della figura. L'immagine di destra mostra come il morsetto si allinea lungo l'asse y e z dando l'impressione all'utente di mantenere l'oggetto in mano. La *figura 25* espone la porzione di codice relativo all'interazione di destra della figura precedente. Alla funzione `Snap()` della classe `SnapInteractorTransformers` si può infatti specificare quale asse di posizione e rotazione escludere;

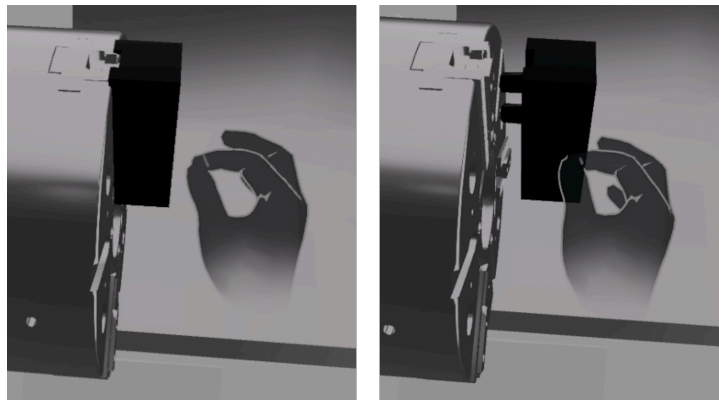


Figura 24: Confronto snap interaction con e senza override della componente orizzontale

```

57     protected override void Snap(Vector3 collisionPoint)
58     {
59         SnapInteractorTransformers.Snap(SocketTransform, collisionPoint, overrideXPosition: false);
60     }

```

Figura 25: Override della funzione Snap della classe Socket

5. **rilascio dell'interactor dal socket** - una volta soddisfatta una determinata condizione definita dal socket, l'interactor lascia l'alloggio e torna libero di muoversi nella mano dell'utente. La condizione può essere semplicemente allontanarsi a sufficienza dal socket, oppure nel caso della chiave a brugola aver compiuto un certo numero di giri per allentare una vite. Al compimento della condizione vengono sostituiti i transformer responsabili delle restrizioni con il FreeGrabTransformer il quale permette un movimento libero. Questa operazione si è rivelata necessaria poiché via codice è possibile solamente assegnare un nuovo transformer, non rimuoverlo o disattivarlo. La classe Grabbable infatti, script utilizzato per afferrare un oggetto, espone solamente i metodi InjectOptionalOneGrabTransformer() e InjectOptionalTwoGrabTransformer(). È stato provato anche a iniettare il valore "null" ma non ha risolto il problema poiché affinché il transformer abbia effetto deve essere inizializzato. La *figura 26* mostra la funzione creata per rimuovere i vincoli;

```

238     private void DisableTransformers()
239     {
240         // adds GrabFreeTransformer if it does not exist or reuses the existing component otherwise
241         if (!_grabbable.TryGetComponent<GrabFreeTransformer>(out var grabFreeTransformer))
242             grabFreeTransformer = _grabbable.AddComponent<GrabFreeTransformer>();
243
244         if (OneGrabTransformer != null)
245         {
246             _grabbable.InjectOptionalOneGrabTransformer(grabFreeTransformer);
247             grabFreeTransformer.Initialize(_grabbable);
248         }
249         if (TwoGrabTransformer != null)
250         {
251             _grabbable.InjectOptionalTwoGrabTransformer(grabFreeTransformer);
252             grabFreeTransformer.Initialize(_grabbable);
253         }
254     }

```

Figura 26: Rimozione dei vincoli dell'oggetto riguardo la snap interaction

1. **aggiornamento del comportamento fisico** - affinché un oggetto non sia affetto da gravità bisogna attivarli la proprietà "isKinematic". Questa permette di manipolare la posizione dell'oggetto anche quando non poggia su alcuna superficie. È utilizzata dagli script della SDK per afferrare gli oggetti e manerli nella mano dell'utente. Una volta rilasciati devono riassumere lo stato iniziale, quindi se normalmente sono affetti da gravità devono cadere. Questo comportamento è desiderato anche per bloccare gli interactor nei socket in modo che non cadano quando l'utente li lascia. Anche l'implementazione

di questa funzionalità ha richiesto diverso tempo perché la SDK assegna l'estensione `RigidbodyKinematicLocker` al `Rigidbody`, il componente del `GameObject` responsabile dei comportamenti legati alla fisica. L'estensione funge da controllore in modo che si possa bloccare o meno la proprietà `isKinematic`. Presenta un contatore che tiene traccia del numero di richieste di bloccare il valore della proprietà. È stata quindi creata una funzione che permettesse di rimuovere tutte le richieste di blocco, assegnare il valore desiderato e reimpostare il contatore al valore iniziale.

Questa tipologia di interazione è stata la più complessa studiata e realizzata; ha fatto emergere diverse considerazioni riguardo la versatilità e manipolabilità della SDK, obiettivo principale dello stage. Tuttavia si è prodotto una interazione fluida, funzionante e scalabile anche grazie all'utilizzo del template method pattern spiegato in dettaglio nella [sezione 5.5.4](#).

5.5. Design pattern

Nella progettazione di prodotto software in genere risultano utili dei design pattern per risolvere in modo efficace e controllato alcuni problemi. Questi aiutano a mantenere una struttura ordinata e manutenibile nel tempo. Nella seguente sezione vengono descritti i design pattern adottati nel progetto.

5.5.1. Dependency injection pattern

Quando un progetto è costituito da un numero considerevole di componenti risulta fondamentale minimizzare le dipendenze. Più si riesce a evitare *debito tecnico*⁴ e più semplice risulta aggiungere funzionalità perché le parti del sistema non sono fortemente accoppiate. L'obiettivo di questo design pattern è quindi quello di togliere a un componente la responsabilità della risoluzione delle proprie dipendenze.

5.5.1.1. Implementazione della dependency injection pattern

Per facilitare l'implementazione del design pattern è stato sfruttato il framework Zenject. Questo mette a disposizione un *Inversion of Control (IoC)*_G definito nella classe `Container` che gestisce i legami tra le dichiarazioni delle dipendenze e le rispettive risoluzioni. È necessario estendere una classe, nel caso del progetto chiamata `AppInstaller`, ad una interna del framework denominata `MonoInstaller` e da questa sovrascrivere il metodo `InstallBindings()` nel quale definire le risoluzioni delle dipendenze. In seguito bisogna assegnare la classe appena creata e lo script `SceneContext`, sempre interno al framework, ad un `GameObject` nella scena. Infine si deve riferire la propria classe all'evento `MonoInstallers` come mostrato in [figura 27](#).

⁴Possibili complicazioni derivanti da scelte progettuali o implementative non giustamente ponderate, in genere per motivi di tempo o risorse.

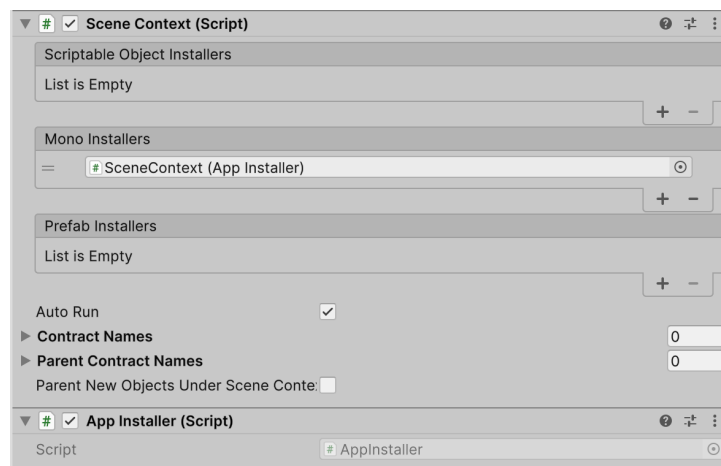


Figura 27: Configurazione degli script di Zenject dall'Editor

Zenject rileva in automatico le dipendenze passate per costruttore e le risolve se trova una corrispondenza nell'IoC. Se invece le dipendenze non sono dichiarate nel costruttore è possibile utilizzare l'attributo `[Inject]` sul campo, proprietà o metodo desiderato.

Nel progetto la maggior parte dipendenze devono essere via Editor in quanto spesso non è richiesto solo una nuova istanza di un oggetto, ma una particolare perché legata a un preciso `GameObject`. Per questo motivo si è fatto largo uso dell'attributo `[SerializeField]` seguito da un campo privato di una classe estesa alla `MonoBehaviour`. In alcuni casi però è risultato utile sfruttare l'IoC di Zenject, in particolar modo per il repository pattern [sezione 5.5.5](#).

I principali concetti utilizzati per risolvere le dipendenze all'interno dell'IoC sono i seguenti:

- * **binding** - definisce con il tipo dell'oggetto con il quale risolvere la dipendenza. Si possono risolvere le dipendenze di classi o anche di interfacce definendo l'implementazione desiderata;
- * **scope** - definisce il tipo di istanza ritornata, suddiviso in tre tipologie:
 - ▶ **transient** - viene restituita una nuova istanza ad ogni richiesta;
 - ▶ **single** - viene restituita la stessa istanza ad ogni richiesta, sfruttando il singleton design pattern.

5.5.2. Singleton pattern

Alcune componenti potrebbero dover mantenere un'integrità per tutta l'esecuzione del prodotto, non possono quindi esistere diverse istanze con diversi valori. Il singleton pattern assicura che ovunque si acceda al componente venga restituita sempre la stessa istanza.

5.5.2.1. Implementazione del singleton pattern

Nel progetto sono stati individuati due metodi per modellare un singleton: utilizzando il metodo `Awake()` di Unity per le classi estese alle `MonoBehaviour` oppure sfruttare gli scope di Zenject. Il primo assegna a una proprietà statica pubblica l'istanza della classe associata al `GameObject`. Inserendo l'operazione all'interno del metodo `Awake()` è facile inoltre controllare se esistono due istanze nella scena, cioè due `GameObject` con associato la classe. Di contro bisogna assicurarsi che il metodo venga richiamato dal processo di istanziamento di Unity prima che un'altra classe acceda alla proprietà statica. In [figura 28](#) è rappresentato l'esempio della manager `MonoBehaviourReference`.

```

3  public class MonoBehaviourReference : MonoBehaviour
4  {
5      4 references
6      public static MonoBehaviourReference Instance { get; private set; }
7
8      @ Unity Message | 0 references
9      private void Awake()
10     {
11         if (Instance == null)
12         {
13             Instance = this;
14             return;
15         }
16         Debug.Log($"A {nameof(MonBehaviourReference)} instance already exists, " +
17             $"the new one will be destroyed.");
18         Destroy(this);
19     }
20 }

```

Figura 28: Singleton implementato in un sottotipo di MonoBehaviour

Nel caso in cui invece la classe possedesse un costruttore proprio, e quindi non è associato ad alcun GameObject, è stato possibile utilizzare il framework Zenject specificando lo scope `AsSingle()` nel binding. Un esempio è fornito in [figura 29](#). Una nota rilevante è legata allo stripping di Unity eliminando le classi che non sono riferite da alcuna parte nella fase di build⁵. Poiché il costruttore non viene mai richiamato Unity, dal momento che le dipendenze sono risolte dall'IoC, la classe verrebbe eliminata. Un modo per evitarlo è contrassegnare un costruttore con l'attributo `[Preserve]` come mostrato in [figura 30](#).

```

5      public override void InstallBindings()
6      {
7          Container.Bind<DataContext>()
8              .AsSingle();
9      }

```

Figura 29: Binding dell'IoC di Zenject per singleton pattern

```

13     [Preserve]
14     0 references
15     public DataContext(CompanyInitializer companyInitializer)
16     {
17         CompanyInitializer = companyInitializer;
18     }

```

Figura 30: Preserve del costruttore per singleton pattern con Zenject

5.5.3. Observer pattern

Quando un componente deve comunicare un cambiamento a altri componenti del sistema può risultare problematico gestire i numerosi riferimenti, rischiando a volte di cadere in una dipendenza circolare⁶. L'observer pattern fornisce una soluzione nella quale i componenti che devono reagire all'aggiornamento si iscrivono al componente desiderato; questo notifica i suoi iscritti quando subisce un cambiamento restituendo il suo nuovo stato. Il pattern adotta interfacce e classi astratte assicurando di evitare la dipendenza circolare.

⁵Fase nello sviluppo nel quale si compila il codice e si genera un eseguibile dell'applicativo.

⁶Situazione nella quale due componenti si riferiscono a vicenda, quindi risulta impossibile istanziarle (a meno di metodi setter) perché ognuna necessita dell'altra.

5.5.3.1. Implementazione dell'observer pattern

C# mette a disposizione una struttura a eventi che sfrutta l'observer pattern. Si crea un evento che gestisce un `EventHandler`, il tipo di dato che l'evento restituisce. Gli ascoltatori si iscrivono all'evento il quale, una volta che viene lanciato, ritorna l'`EventHandler`. L'esempio in [figura 31](#) e [figura 32](#) mostra l'evento personalizzato dello step che notifica alla procedura di essere stato completato, fornendo il proprio indice di ordine di esecuzione.

```
6      public event EventHandler<OnStepCompletedEventArgs> OnStepCompleted;  
7      5 references  
      public class OnStepCompletedEventArgs : EventArgs  
8      {  
9          4 references  
          public uint Index { get; set; }  
10     }
```

Figura 31: Dichiarazione dell'evento step completato nella classe Step

```
67     foreach (var step in _stepArray)  
68     {  
        step.OnStepCompleted += StepInProcedure_OnStepCompleted;  
    }
```

Figura 32: Iscrizione all'evento step completato nella classe Procedure

5.5.4. Template method pattern

Un template è una struttura che viene ripresa in più istanze e arricchita a seconda delle esigenze. Il componente che funge da template mette quindi a disposizione dei metodi finali, che non possono cioè essere sovrascritti, e dei metodi astratti che devono essere definiti dalle classi che ereditano il template. I primi corrispondono alle caratteristiche comuni ai derivati del componente template, i secondi invece alle proprietà caratterizzanti le singole istanze.

5.5.4.1. Implementazione del template method pattern

Il design pattern è risultato di fondamentale importanza per strutturare, in primo luogo, l'architettura della snap interaction ([sezione 5.4](#)). L'obiettivo di questa è assegnare dei vincoli di movimento agli oggetti tenuti in mano dall'utente quando questi vengono avvicinati a un alloggio chiamato socket. Rifacendosi alla [sezione 2.2.3.1](#) gli oggetti sono gli interactor mentre i socket sono gli interactable. Per entrambi i componenti è presente una logica che consente l'interazione tra di essi e con l'utente, mentre le restrizioni sul movimento e le condizioni per le quali applicare i vincoli o toglierli sono specifici di ciascun componente.

Ambo le classi `SnapInteractorTransformers` e `Socket` sono quindi astratte e estese alla `MonoBehaviour` in modo che le derivate possano essere associate a un `GameObject`. Per creare un nuovo interactor e il rispettivo socket è sufficiente derivare le classi template, definire i metodi astratti e associare i nuovi script agli oggetti nella scena. La [figura 33](#) espone il diagramma della classe `Socket`; alcuni campi e i sottotipi vengono omessi per semplicità.

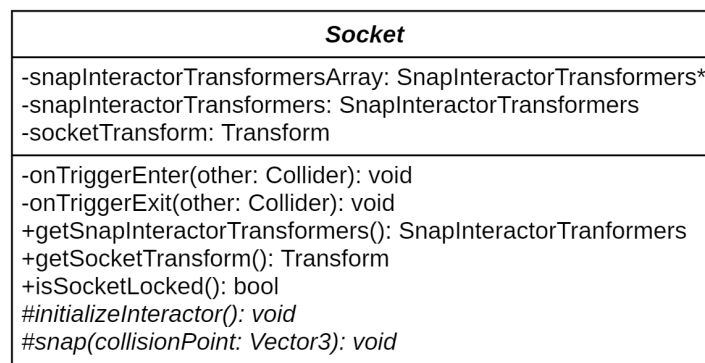


Figura 33: Diagramma della classe Socket

L'altra casistica nella quale il template method pattern è risultato conveniente è stato riguardo la struttura degli step. La classe Step contiene le proprietà comuni quali indice (ordine all'interno della procedura), descrizione, istruzione al completamento e l'evento da lanciare quando il passaggio è completato; i metodi astratti invece riguardano l'aggiornamento degli stati degli elementi. La *figura 34* ne espone il diagramma della classe; i sottotipi vengono omessi per semplicità.

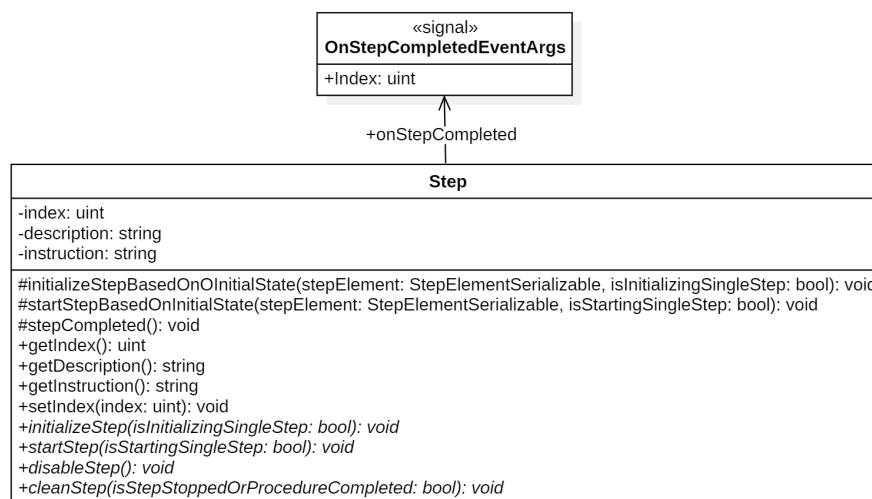


Figura 34: Diagramma della classe Step

5.5.5. Repository pattern

La gestione della persistenza dei dati è un punto cruciale di ogni applicazione che non dovrebbe dipendere dal sistema di base di dati utilizzato. Il metodo di persistenza di dati, sia esso un database relazionale, non relazionale, un file o un sistema esterno al quale si acceda tramite chiamate API, non dovrebbe incidere sulla *business logic*. Prima di indagarne l'aspetto strutturale è importante comprendere la divisione tra dati “grezzi” e risorse: i primi sono dati semplici come stringhe e numeri, immediati da reperire, i secondi sono informazioni dal peso rilevante come foto, video, effetti audio o visivi. Spesso non è desiderabile caricare tutti i dati all'avvio dell'applicazione perché potrebbe rallentare le altre operazioni, quindi si procede a recuperare prima i dati e quando necessario le risorse. Inoltre mantenendo la separazione è possibile utilizzare due differenti metodi di recupero. Il design pattern prevede quindi di ritirare prima i dati ed elaborarli in modo da generare un “model”. In seguito si utilizzano alcuni dati del model per recuperare le risorse, per esempio l'indirizzo al quale fare la richiesta API. Una

volta scaricate anche queste vengono elaborate e viene creato un “view model”, il quale poi è restituito alla business logic.

La separazione tra dati e risorse può essere mantenuta anche all'interno del repository, creando cioè due classi separate: una addetta alla creazione del model, l'altra a quella del view model. La *figura 35* illustra uno schema del repository pattern.

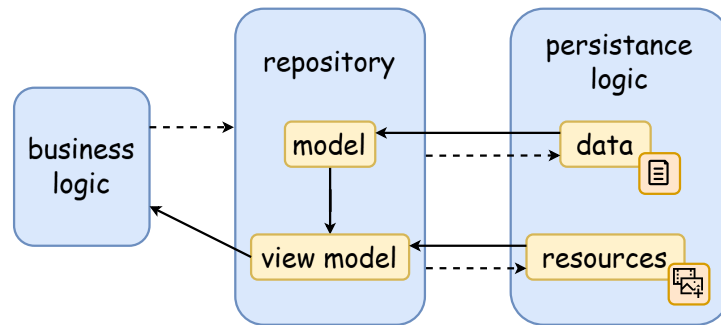


Figura 35: Schema logico del repository pattern

5.5.5.1. Implementazione del repository pattern

Seppur in maniera semplificata, è stato deciso di dimostrare l'attuabilità di persistere delle informazioni in locale sul dispositivo e recuperarle quando necessario. A tale scopo sono stati memorizzati alcuni dati dell'azienda in un file .json e il logo della stessa in formato .png. Il componente principale dell'applicazione, chiamato MainController, richiede l'inizializzazione del DataContext il quale lancia un evento al termine dell'operazione. La classe DataContext contiene lo stato dell'applicazione riguardo i dati da recuperare, quindi il view model da generare e l'evento per notificare l'avvenuta inizializzazione. La classe utilizzata una istanza di CompanyInitializer il quale contiene i riferimenti ai due repository: CompanyDataRepository addetto al recupero dei dati dal file .json e alla generazione del CompanyModel; CompanyResourceRepository incaricato di utilizzare il percorso fornito dal model per recuperare il logo in formato .png, creare uno *sprite* e restituire il CompanyViewModel al CompanyInitializer, quindi poi al DataContext. I metodi dei repository utilizzano una *callback* di successo e una di errore per segnalare al chiamante il risultato dell'operazione.

In questo modo l'applicazione, una volta che il DataContext è inizializzato, utilizza dei tipi di dato che sa interpretare, cioè stringhe e sprite. La *figura 36* espone il diagramma delle classi della struttura appena descritta.

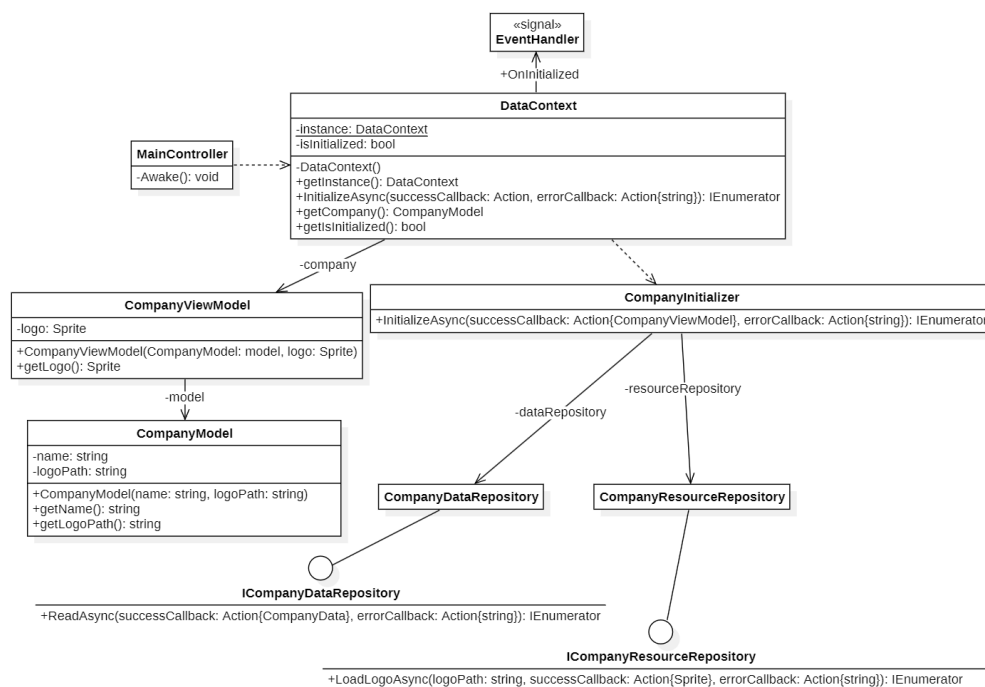


Figura 36: Diagramma delle classi relative al repository pattern

La *figura 37* mostra i dati del file .json. La *figura 38* corrisponde alla richiesta, da parte di CompanyInitializer, di recuperare i dati e alla successiva inizializzazione del model in caso di successo. La *figura 39* invece espone il codice riguardo la richiesta di recuperare il logo sotto forma di sprite e, in caso di successo, la creazione del view model. Questo poi viene restituito al chiamante, ovvero DataContext, tramite la callback di successo.

```

1  {
2      "name": "Airlapp",
3      "logoLocalPath": "AirlappLogo.png",
4      "address": "Via S. Pio X, 49/1, Piove di Sacco (PD)",
5      "mail": "info@airlapp.com"
6  }

```

Figura 37: Dati del file .json

```

26  // read company data
27  yield return DataRepository.ReadAsync(
28      // success callback
29      (companyModel) => model = companyModel,
30
31      // error callback
32      (error) => errorCallback(error)
33  );

```

Figura 38: Creazione del model

```
41         // load company view model from model
42         yield return ResourceRepository.LoadLogoAsync(
43             model.LogoPath,
44
45             // success callback
46             (logo) =>
47             {
48                 var companyViewModel = new CompanyViewModel(model, logo);
49                 successCallback(companyViewModel);
50             },
51
52             // error callback
53             (error) => errorCallback(error)
54         );
```

Figura 39: Creazione del view model

5.5.6. Command pattern

Alcuni componenti potrebbero dover svolgere diverse azioni a seconda della condizione in cui si trovano. Si può prendere come esempio una situazione dell'applicazione: la console con la quale si interagisce per operare sul macchinario contiene numerosi pulsanti e uno schermo; tramite il display è possibile selezionare un componente mobile del macchinario e muoverlo utilizzando le freccette sulla tastiera. A seconda del contesto, quindi quale componente del macchinario è stata scelta, le freccette devono compiere azioni diverse. Il command pattern attribuisce alle classi interessate, in questo caso quelle delle freccette, un comando ragionando a livello astratto di interfaccia. In questo modo il contesto esterno può fornire il proprio comando da eseguire.

5.5.6.1. Implementazione del command pattern

Come anticipato il command pattern è risultato estremamente importante per programmare i pulsanti e le azioni della console. In particolare sono state utilizzate due interfacce: `IButtonCommand` per i comandi relativi alle freccette, `ITunerCommand` per quelli relativi alla manopola di precisione. In base alla selezione dell'utente la classe `Console` imposta l'implementazione di un comando. All'interazione con i componenti programmati, in questo caso le freccette e la manopola, vengono eseguiti i comandi precedentemente impostati. In [figura 40](#) viene illustrata una parte della struttura del design pattern.

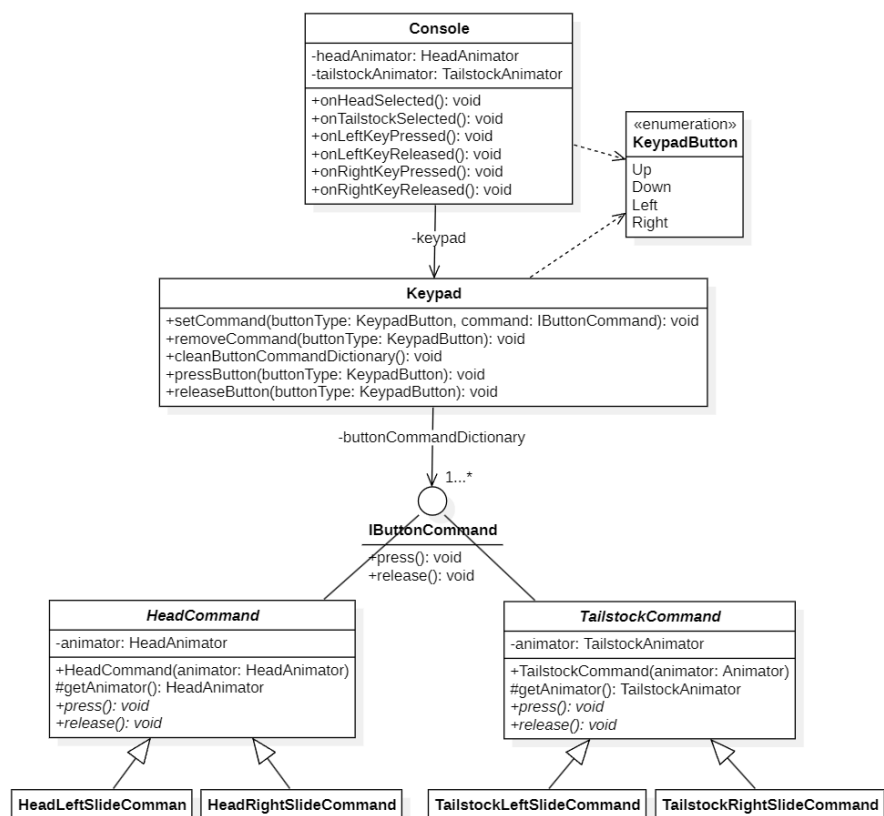


Figura 40: Diagramma di alcune classi relative al command pattern

Capitolo 6

Verifica e validazione

Di per sé l'ambiente di sviluppo Unity non permette di implementare, e di conseguenza eseguire, test automatici durante il processo di sviluppo di un applicazione. Allo stato attuale essi possono essere eseguiti attraverso l'utilizzo di framework, i quali tuttavia risultano ancora acerbi e presentano alcune limitazioni. I singoli componenti e le relative funzionalità sono quindi state verificate manualmente tramite l'esecuzione dell'applicazione sul visore. Inoltre è risultato fondamentale l'utilizzo del package Android Logcat che ha permesso di monitorare e filtrare gli avvisi di debug lanciati durante l'esecuzione.

Infine sono stati definiti alcuni test di accettazione che hanno permesso di validare l'applicazione finale. Nella *tabella 7* sono inseriti i test eseguiti.

Tabella 7: Test di accettazione

Funzionalità	Esito	Fonti
Movimento nello spazio virtuale.	Superato	<u>UC1</u> e inclusioni
Interazioni eseguibili sia con i joypad che con le mani.	Superato	<u>UC2.1</u> <u>UC2.2</u> <u>UC3.1</u> <u>UC3.2</u> <u>UC4.1</u> <u>UC4.2</u> <u>UC5.1</u> <u>UC5.2</u>
Almeno una interazioni per tipo tra le seguenti: * traslazione di un oggetto; * rotazione di un oggetto; * ancoraggio automatico di un oggetto in un alloggio; * selezione di un elemento UI.	Superato	<u>UC2</u> <u>UC3</u> <u>UC4</u> <u>UC5</u>
Istruzioni per lo svolgimento delle attività, sia su interfaccia grafica sia nella scena.	Superato	<u>UC13.1</u> <u>UC13.2</u> Confronto con l'azienda
Menu principale per la selezione delle attività.	Superato	<u>UC7</u> e inclusioni <u>UC8</u> e inclusioni
Stabilità degli <i>Frame Per Second (FPS)</i> _G a un minimo di 20.	Superato	Confronto con l'azienda

Capitolo 7

Conclusioni

Al termine del progetto sono emerse le potenzialità della realtà virtuale in ambito formativo, obiettivo primario dello stage. Con la continua e rapida evoluzione della SDK di Meta è evidente la versatilità delle componenti fornite per comporre nuove interazioni tra uomo e ambiente virtuale. In alcuni ambiti la tecnologia è ancora acerba e priva di documentazione adeguata, difficile quindi da sfruttare appieno. Con alcuni accorgimenti è possibile trovare delle soluzioni in principio non ottimali, che quindi rischiano di aumentare il debito tecnico, ma che si rivelano fondamentali per l'attività di studio e ricerca.

Al fine di concludere con un prodotto software completo e utilizzabile, l'attività di studio e ricerca è stata calata in un contesto reale di formazione di personale neoassunto addetto all'utilizzo e manutenzione di macchinari specializzati. Nello specifico è stato trattato un tornio per materiali di grandi dimensioni. La *figura 41* mostra quello inserito nell'applicazione dove sono stati omessi marca e modello per questione di privacy aziendale.



Figura 41: Macchinario utilizzato inserito nella scena virtuale

Le interazioni indagate sono state distribuite in maniera abbastanza uniforme su tutte le operazioni eseguibili all'interno dell'applicazione:

- * un menu iniziale dal quale scegliere l'attività da svolgere (*figura 42*) e un pannello informativo all'interno della scena (*figura 43*) forniscono la possibilità di interagire con delle interfacce grafiche. Con la tipologia di UI scelta è permesso di interagire solamente da distante, utilizzando cioè i controller o le mani come fossero dei telecomandi puntando e selezionando gli elementi desiderati;



Figura 42: Visuale del menu nella scena virtuale

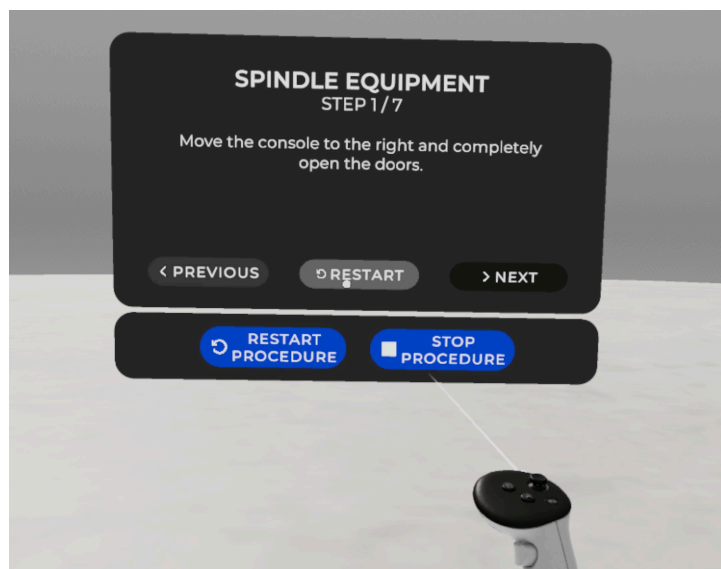


Figura 43: Visuale del pannello informativo nella scena virtuale

- * le porte e la console del macchinario scorrono orizzontalmente lungo delle corsie apposite. Possiedono infatti dei maniglioni con i quali spostarli. Sono state quindi aggiunte delle interazioni adatte a questo particolare movimento. Inoltre alcuni oggetti come i morsetti, la punta di incisione e il materiale da installare nel mandrino⁷ devono essere infilati negli appositi alloggi. L'operazione di installazione richiede quindi un simile movimento di traslazione. La *figura 44* mostra la visuale dell'interazione con la console e la punta da intaglio;

⁷Dispositivo meccanico che permette di serrare un pezzo in lavorazione, nonché di trascinare in rotazione un utensile.



Figura 44: Traslazione della console e della punta da intaglio

- * simile all'interazione precedente è stata inserita la possibilità di ruotare un oggetto attorno a un asse. L'implementazione ha riscontrato diverse difficoltà in quanto gli script forniti dalla SDK presentano ancora alcune fallacie, aspetto affrontato in maggior dettaglio nella sezione 7.3.1. È stato comunque possibile implementare i meccanismi di rotazione di una chiave a brugola, una manopola e un mandrino. I primi due sono mostrati in figura 45;

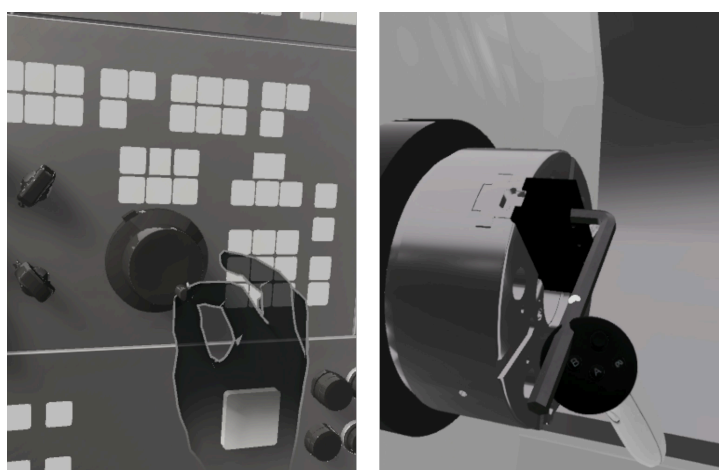


Figura 45: Rotazione della chiave a brugola e della manopola di precisione

- * la snap interaction è stata quella più complessa da realizzare ma è risultata fondamentale per gran parte delle interazioni. La maggior parte degli oggetti infatti dovevano essere inseriti in particolari alloggi: i morsetti negli appositi incastri del mandrino, la chiave a brugola nelle viti dei morsetti, il materiale grezzo nel mandrino e la punta da intaglio nell'apposita testa rotante;
- * l'ultima tipologia di interazione comprende i pulsanti della console. Questi infatti devono essere premuti con la parte frontale del controller o con un dito della mano. Il pulsante è presente in una posizione di rilascio, mentre retrocede fino a un certo limite quando l'utente va a premerlo. Non presenta posizioni predefinite, segue il movimento dell'interactor, controller o mano che sia. Unica eccezione è stata fatta per il pulsante di emergenza il quale, una volta premuto, deve rimanere bloccato fino alla messa in sicurezza del macchinario. La figura 46 mostra due esempi dell'interazione.

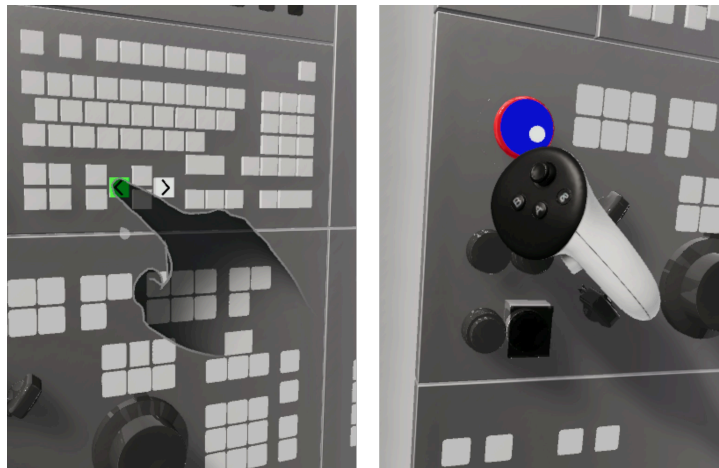


Figura 46: Pressione del pulsante di emergenza e di una freccetta

7.1. Raggiungimento degli obiettivi

L'azienda ha concesso diverso spazio di manovra tracciando a inizio tirocinio solamente delle linee guida da seguire. L'obiettivo principale dello stage è stato infatti quello di studio delle interazioni. Sebbene fosse calato in un contesto nel quale dovevano essere operative delle procedure ben strutturate, attività che ha richiesto circa le ultime tre settimane di lavoro, l'attenzione principale è stata rivolta alle possibili combinazioni di funzionalità fornite dalla SDK di Meta. Circa i due terzi del periodo trascorso infatti è stato incentrato a capire e sfruttare le componenti e gli script messi a disposizione dalla SDK. Dopo i primi approcci con le interazioni infatti si è passati alla composizione di nuove strutture più complesse quali torsioni degli oggetti, spostamenti di precisione e limitazioni a mirati movimenti in determinate circostanze. In contemporaneo quindi è stata svolta l'analisi dei requisiti fino a produrre un *Proof of Concept (PoC)* e capire, nei limiti del periodo di tirocinio, quali funzionalità si sarebbero potute concludere.

In linea generale sono comunque stati raggiunti tutti gli obiettivi prefissati nel Piano di Lavoro a eccezione dell'**F2**. La *tabella 8* riassume gli obiettivi con il rispettivo esito.

Tabella 8: Tracciamento degli obiettivi raggiunti

Codice	Descrizione	Esito
O1	Studio e analisi della tecnologia di tracciamento delle mani sostitutiva ai joypad nella applicazione prototipale.	Raggiunto
O2	Definizione degli scenari operativi su simulatore VR.	Raggiunto
O3	Studio e implementazione della UI per il VR.	Raggiunto
O4	Implementazione degli scenari operativi su simulatore VR.	Raggiunto
O5	Sviluppo della applicazione prototipale su framework Unity.	Raggiunto
O6	Testing del sistema su Meta Quest 3.	Raggiunto
O7	Redazione di una documentazione tecnica e metodologica per il progetto pilota.	Raggiunto
D1	Monitoraggio del progresso formativo.	Raggiunto
D2	Integrazione di funzionalità di feedback aptico o audio per migliorare il realismo dell'esperienza.	Raggiunto

Codice	Descrizione	Esito
F1	Ampliamento delle procedure e delle interazioni utente di simulazione.	Raggiunto
F2	Ottimizzazione delle prestazioni per garantire fluidità ed esperienza immersiva.	Non raggiunto

Al termine del processo di sviluppo sono stati assegnati materiali, texture e luci all'ambiente virtuale. Queste operazioni sono quelle che maggiormente impattano sulle performance dell'applicazione, specialmente su un visore di realtà virtuale la quale potenza di calcolo è limitata. Un problema inoltre riscontrato fin dal principio del progetto è stata la densità poligonale⁸ del modello 3D. Nella fase finale del progetto è stata monitorata la variazione degli FPS utilizzando l'applicazione OVR Metrics Tool. In seguito a numerosi tentativi di ridurre il peso grafico dell'applicazione si è riuscito a raggiungere una stabilità tale da non causare, almeno tra le persone che hanno provato il visore, sensi di nausea dovuti alla bassa frequenza delle immagini proiettate. È inoltre importante sottolineare che i visori non permettono di raggiungere livelli di performance paragonabili ad applicazioni su computer o mobile. L'obiettivo **F2** tuttavia è considerato non raggiunto in quanto, con maggiore tempo e risorse, si è pensato si possano raggiungere performance più elevate.

7.2. Soddisfacimento dei requisiti

Una corretta distribuzione del tempo e delle attività ha portato al soddisfacimento della totalità dei requisiti elencati nella *sezione 4.3*, anche quelli desiderabili e facoltativi. Pertanto non viene riportata una tabella per il tracciamento. L'assistenza dell'intero team dell'azienda è risultato fondamentale a completare alcuni aspetti più complessi legati alla progettazione e sviluppo in tempi ragionevoli, permettendo di avanzare con il lavoro senza significative interruzioni.

7.3. Considerazioni personali

L'esito finale dell'esperienza è sicuramente positivo. Il tema proposto dall'azienda è molto innovativo e ricercato, caratterizzato da esperienze nuove sia lato sviluppatore che lato utente. Un semplice esempio è stato il mancato utilizzo delle periferiche classiche di un computer per il testing dell'applicazione. Provenendo da una esperienza quadriennale di sviluppo videogiochi è stato facile orientarsi con le nuove tecnologie, tuttavia non avevo mai lavorato con la realtà virtuale. Approfondire alcuni aspetti dell'engine e verificarli in un ambiente virtuale immersivo è risultato molto stimolante.

Lo scopo principale, ovvero indagare la versatilità delle interazioni, è divenuto col passare dei giorni fonte di curiosità e personale interesse. Al compimento di una piccola funzionalità, per esempio afferrare un oggetto, si dialogava con i responsabili dell'azienda circa il modo per arricchirla oppure per studiarne una nuova. Questo processo è terminato unicamente per questioni di tempo dopo circa 5 settimane.

La *figura 47* fornisce la visuale dell'utente all'interno della scena virtuale.

⁸Nel caso attuale inteso come numero di poligoni per unità di mesh. Un numero elevato di triangoli da renderizzare in poco spazio aumenta considerevolmente la quantità di calcoli da produrre.



Figura 47: Visuale completa della scena virtuale

7.3.1. Aspetti implementati e possibili integrazioni

La discreta libertà di azione concessa dall'azienda ha permesso di studiare e implementare gli aspetti che più ritenevo interessanti da un punto di vista tecnico. Per quanto riguarda le interazioni i risultati sono stati incrementali in quanto le conoscenze acquisite da alcune hanno portato a comporre quelle successive. Tuttavia al termine del periodo dedicato all'implementazione delle interazioni si è dovuto abbandonare due aspetti che avrebbero richiesto diverso tempo: il primo è l'utilizzo delle hand pose, permettere cioè alla mano virtuale di assumere una determinata posa afferrando un oggetto. Il building block utilizzato per generare il Camera Rig non definiva correttamente i componenti per permettere questo tipo di interazione e avendo usato la versione della SDK più recente, al momento del progetto la 76, la documentazione e i forum online non sono stati sufficienti a risolvere il problema. La seconda funzionalità non implementata è stata la rotazione di un oggetto, sempre afferrato dall'utente, limitato in un determinato angolo d'azione. Sarebbe stato immediato associare lo script `OneGrabRotateTransformer` fornito dalla SDK, tuttavia inaspettatamente le limitazioni si invertivano di segno al reset della scena. Quando cioè lo step veniva completato si ruotava l'oggetto allo stato iniziale ma da quel momento i vincoli erano invertiti, passando cioè dall'intervallo 0° - 90° a 270° - 0° . Per lo stesso motivo della versione della SDK sono stati trovati comportamenti simili ma nessuna soluzione e averne prodotta una personalmente avrebbe richiesto troppo tempo.

Sul lato progettuale invece l'azienda si è mostrata disponibile in modo da approfondire aspetti non affrontati durante i corsi universitari. Particolare attenzione è stata data al backend e al repository pattern. Dal momento che non rientrava negli obiettivi dello stage si è voluto implementare una struttura piuttosto semplice, ma sarebbe stato istruttivo completarla aggiungendo ad esempio un effettivo sistema di base di dati al posto del file .json, oppure implementando il "lazy loading", tecnica che consiste nel caricare una risorsa solo quando viene richiesta e non all'avvio della applicazione. Un backend più completo avrebbe potuto inoltre permettere di strutturare un metodo di caricamento della scena più dinamico ed efficiente, rendendo peraltro automatici i riferimenti tra i GameObject.

7.3.2. Conoscenze e competenze acquisite

L'esperienza dello stage dal punto di vista formativo è da considerare estremamente positiva. Le conoscenze pregresse relative al framework Unity hanno permesso di cominciare fin dall'inizio ad approfondire la SDK, fornendo in seguito sufficiente spazio alla fase progettuale. Non si sono arricchite quindi tanto le conoscenze quanto le competenze. Si arricchito sicuramente il bagaglio tecnico approfondendo alcuni aspetti di Unity, vedendo nel dettaglio il repository pattern mai affrontato durante le lezioni e discutendo di alcuni metodi di ottimizzazione delle applicazioni.

Più di questo tuttavia sono aumentate le competenze riguardo saper consultare e sfruttare una SDK e portare a compimento diversi aspetti progettuali di una applicazione complessa. Tutta la parte architettuale infatti, a eccezione del backend, è stata strutturata senza importanti consultazioni con l'azienda mettendo in pratica le conoscenze acquisite durante il tirocinio e i corsi universitari.

Oltre ad aspetti tecnici informatici è stata appresa anche l'importanza del rispetto delle scadenze. Nonostante non ci fossero dei requisiti ben precisati sulla quantità e tipologia delle interazioni è stato comunque fondamentale definire delle scadenze interne da rispettare in modo da avere sufficiente tempo e risorse per concludere il progetto nel modo più completo possibile. La padronanza dell'IDE di sviluppo e del linguaggio di programmazione ha sicuramente aiutato a calibrare con maggiore precisione gli impegni delle singole attività. Un metodo di lavoro organizzato ha portato quindi a concludere il progetto secondo le aspettative iniziali con soddisfazione sia da parte dell'azienda che da parte mia.

Glossario

Assembly Unità fondamentale per il rilascio di una applicazione, il suo versionamento, riutilizzo e permessi di sicurezza. In .NET sono come dei pacchetti che contengono le risorse "assemblate" necessarie per avviare l'eseguibile o le librerie. [7](#).

Business logic Parte centrale di una applicazione che ne cura il comportamento logico. Dialoga con l'application logic, che si interfaccia con l'utente, e la persistence logic, responsabile del mantenimento dei dati.. [44](#), [45](#).

Callback Operazioni che il chiamante passa come parametro in modo che vengano eseguite al raggiungimento di alcune situazioni, ad esempio in caso di successo o di errore. [45](#), [46](#).

Debito tecnico Possibili complicazioni derivanti da scelte progettuali o implementative non giustamente ponderate, in genere per motivi di tempo o risorse. Con l'evolvere dell'applicazione rendono più difficoltoso il processo di manutenzione del software. [40](#), [50](#).

GameObject Oggetto della scena in Unity. Sono formati dalla composizione di vari componenti i quali attribuiscono le proprietà desiderate all'oggetto. [7](#), [10](#), [11](#), [12](#), [33](#), [38](#), [40](#), [41](#), [42](#), [43](#), [55](#).

IntelliSense Insieme di funzionalità volte ad agevolare la scrittura di codice: pannelli rapidi per visualizzare informazioni riguardanti classi, metodi e attributi, lista di funzioni disponibili per una classe, tracciamento di dove vengono usati classi, metodi e proprietà, auto-completamento delle parole. [6](#).

IoC Principio della progettazione software per il quale un componente riceve le proprie dipendenze risolte da una sorgente esterna. [40](#), [41](#), [42](#).

Mesh Insieme di poligoni, solitamente triangoli, disposti in uno spazio tridimensionale. Gli spigoli sono congiunti in modo da formare una superficie sfaccettata. [8](#), [54](#).

Package Insieme di risorse pubbliche riutilizzabili nel proprio progetto di Unity. Alcuni sono fondamentali come l'Input System, Timeline, Unity UI e quelli relativi al rendering (nel progetto è stata adottata la Universal Render Pipeline); molte altre sono opzionali e si possono installare direttamente dall'Editor. [11](#), [12](#), [49](#).

Prefab GameObject salvato negli Asset di Unity. Non dipende dunque dalla scena nella quale viene inserito bensì può essere riutilizzato e duplicato. Le sue istanze risentono dei cambiamenti dell'asset, quindi modificando il prefab si aggiornano tutte le sue istanze. [11](#), [12](#), [13](#), [34](#).

Scenari operativi Operazioni costituite da delle interazioni con l'ambiente, come ad esempio l'utilizzo di una interfaccia utente. [2](#), [3](#), [53](#).

Script Documento contenente codice, nel progetto tipicamente in C#. [10](#), [12](#), [33](#), [38](#), [39](#), [40](#), [43](#), [52](#), [53](#), [55](#).

Sprite Speciale texture generata da una immagine. Si applica a oggetti bidimensionali come le immagini su una interfaccia grafica. [45](#), [46](#).

Thread Suddivisione di un processo in due o più sottoprocessi che vengono eseguiti concorrentemente. [7](#), [33](#).

Acronimi e abbreviazioni

- API** Application Programming Interface. 3, 7, 32, 44.
- CLI** Common Language Infrastructure. 7.
- CLR** Common Language Runtime. 7.
- CTS** Common Type Specification. 7.
- FPS** Frame Per Second. 49, 54.
- IDE** Integrated Development Environment. 6, 10, 56.
- IL** Intermediated Language. 7.
- IoC** Inversion of Control. 40, 41, 42.
- JIT** Just In Time. 7.
- LTS** Long-Term Support. 10.
- PoC** Proof of Concept. 53.
- RAD** Rapid Application Development. 6.
- SDK** Software Development Kit. 7, 10, 12, 34, 38, 39, 40, 50, 52, 53, 55, 56.
- UI** User Interface. 33, 49, 50, 53.

Sitografia

Il Sensorama di Morton Heilig

<https://osservatoriometaverso.it/sensorama-la-realta-virtuale-degli-anni-60>

Il visore 'La Spada di Damocle' di Ivan Sutherland

<https://www.metacommunications.it/metaverso/origini-della-realta-virtuale-che-andrebbero-insegnate>

Primi passi con Meta Quest 3

<https://www.meta.com/it-it/help/quest/1994971530885728>

Unity real-time development platform for VR applications

<https://unity.com/solutions/vr>

Meta Horizon SDK for Unity

<https://developers.meta.com/horizon/develop/unity>

Framework Zenject per la gestione della dependency injection

<https://github.com/modesttree/Zenject>

Struttura e funzionamento di un tornio

<https://it.wikipedia.org/wiki/Tornio>